

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Архитектура суперкомпьютерных систем

Курсовая работа

Студент: _____

Салимли Айзек Мухтар Оглы

Преподаватель: _____

Чуватов Михаил Владимирович

«_____» _____ 20__ г.

Санкт-Петербург, 2026

РЕФЕРАТ

Объем отчета: 73 с., 2 табл., 7 источн., 8 прил.

Ключевые слова: Hyper-V, CUDA, OpenMPI, GPU-ускорение, HPC, виртуальный кластер, параллельное программирование.

Задача исследования - разработка виртуального вычислительного кластера на базе гипервизора Hyper-V с поддержкой GPU-ускорения и создание параллельного приложения для анализа данных с использованием гибридной модели распределенных вычислений.

В процессе работы были настроены четыре виртуальных узла (два CPU-узла и два гетерогенных (с GPU) узла) под управлением Ubuntu Server 22.04, сконфигурирована система управления рабочей нагрузки **SLURM**, развернута серверная файловая система NFS для совместного доступа к данным, реализована поддержка CUDA для GPU-ускорения вычислений. Разработано параллельное приложение для анализа датасета транзакций, которое выполняет фильтрацию записей ($\text{Amount} \geq 9000$), группировку по интервалам признака V1 с шагом 0.1 и формирует итоговый отчет. В отчете выводятся 50 групп с наибольшим математическим ожиданием V11 среди групп с максимальным числом отрицательных значений V7, отсортированные по убыванию общей суммы Amount в группе.

Анализ результатов показал, что оптимальная конфигурация кластера (2 CPU + 2 GPU), обеспечивает ускорение производительности в N раз по сравнению с одноядерной реализацией. Эффективность распределения вычислительной нагрузки достигается через динамическую балансировку с использованием весов (параметра) отношения производительности GPU к CPU, подбор которого позволяет минимизировать простой вычислительных узлов.

Практическая ценность полученных результатов заключается в успешной демонстрации возможности создания высокопроизводительных вычислительных систем на базе виртуальных машин с использованием доступного оборудования. Разработанные в рамках работы алгоритмы распределённой обработки пространственных данных и балансировки нагрузки в гетерогенных кластерах представляют практическую ценность для решения широкого круга задач анализа данных. Кроме того, вся созданная программно-аппаратная среда может быть эффективно использована в учебном процессе для изучения современных технологий параллельного и распределённого программирования.

Содержание

Термины и определения	5
Введение	7
1 Постановка задачи	9
2 Настройка среды разработки	10
2.1 Конфигурация Hurer-V	10
2.1.1 Установка и подготовка Hurer-V	10
2.1.2 Конфигурация основных параметров Hurer-V	10
2.2 Настройка сети	11
2.3 Настройка виртуальной машины	11
2.4 Настройка параметров виртуальной машины	12
2.5 Установка операционной системы	13
2.6 Клонирование виртуальных машин	14
2.6.1 Запуск служб	15
2.7 Настройка серверной части (NFS и CUDA)	15
2.7.1 NFS-шеринг	15
2.7.2 Проверка пакетов CUDA	16
3 Проверка работоспособности кластера	17
3.1 Проверка работоспособности CUDA	17
3.2 Проверка MPI	17
4 Анализ поставленной задачи и подходы к решению	19
4.1 Постановка задачи анализа данных	19
4.2 Неоптимальные подходы к решению	19
4.2.1 Подход 1: Синхронная модель с последовательными барьерами	19
4.2.2 Подход 2: Статическое распределение данных без учёта производительности GPU	20
4.2.3 Подход 3: Блокирующая отправка результатов с ожиданием главного узла	20
4.3 Финальное решение: Гибридная параллельная архитектура	20
4.3.1 Подход 3: Последовательная обработка с барьерами синхронизации	20
4.4 Финальное решение: Гибридная параллельная архитектура	21
4.4.1 Динамическое распределение нагрузки	21
4.4.2 Гибридная обработка: MPI + CUDA	21
4.4.3 Асинхронная агрегация результатов	21
4.4.4 Оптимизированная GPU-реализация	22
4.4.5 Этапы работы финального решения	22
4.5 Пояснение выбора решения	23
5 Анализ датасета транзакций	24
5.1 Общее описание датасета	24
5.2 Структура и описание колонок	24
5.3 Пример записи датасета	25
5.4 Особенности обработки для поставленной задачи	25
6 Разработка программного средства	27
6.1 Архитектура программного средства	27
6.2 Описание структур данных	27
6.3 Динамическое распределение вычислительных ресурсов	27
6.4 Параллельное чтение входных данных	28
6.5 Локальная агрегация и GPU-ускорение	28

6.6	Сбор и редукция интервалов через MPI	28
6.7	Финальная обработка и вывод результатов	29
6.8	Проблемы, выявленные при выполнении работы	29
6.8.1	Проблема статического распределения ресурсов	29
6.8.2	Проблема выбора корректной модели параллелизации	30
6.8.3	Подход 2: Статическое распределение данных без учёта производительности GPU	30
6.8.4	Подход 3: Блокирующая отправка результатов с ожиданием главного узла	30
6.8.5	Подход 4: Последовательная обработка с барьерами синхронизации	31
6.9	Финальное решение: гибридная параллельная архитектура с динамическим распределением	31
6.9.1	Идея параллелизации в финальной реализации	31
6.10	Особенности реализации	31
6.10.1	Особенности GPU-агрегации и сортировки	32
6.10.2	Балансировка нагрузки и отказ от OpenMP	32
6.10.3	Правка весов	32
7	Результаты	33
8	Заключение	35
	Приложение А	37
	Приложение В	38
	Приложение С	40
	Приложение D	42
	Приложение E	44
	Приложение F	48
	Приложение G	52
	Приложение H	66
	Приложение I	72
	Приложение J	73

Термины и определения

В отчете применяют следующие термины с соответствующими определениями:

Таблица 1: Термины и определения

Термин	Определение
Виртуальная машина	Программная эмуляция физического компьютера, позволяющая запускать операционную систему в изолированной среде на физическом хосте.
Виртуальный кластер	Группа виртуальных машин, объединённых в единую вычислительную систему для выполнения распределённых задач.
Вычислительные ресурсы	Аппаратные и программные компоненты компьютерной системы, используемые для выполнения задач: процессоры, память, дисковое пространство, сетевые и графические устройства.
Гипервизор [5]	Программный слой виртуализации, обеспечивающий создание и управление виртуальными машинами на физическом оборудовании.
Графический процессор (GPU)	Специализированный вычислительный блок, оптимизированный для выполнения параллельных вычислений и обработки графики.
Контрольные точки (Checkpoints)	Функция Nureg-V, позволяющая сохранять состояние виртуальной машины в определённый момент времени для последующего восстановления.
Операционная система	Программное обеспечение, управляющее аппаратными ресурсами компьютера и предоставляющее интерфейс для выполнения приложений.
Программное обеспечение	Набор программ и данных, предназначенных для выполнения определённых функций на вычислительных устройствах.
Процессор (CPU)	Центральное вычислительное устройство компьютера, выполняющее основные задачи обработки данных и управления системой.
Сеть	Совокупность взаимосвязанных вычислительных устройств, обеспечивающих обмен данными и совместное использование ресурсов.
Узел	Логическая единица вычислительной системы, представляющая собой физический или виртуальный компьютер с выделенными ресурсами для выполнения задач.
CUDA [3]	Программная платформа компании NVIDIA для организации параллельных вычислений на графических процессорах (GPU).
MPI [2]	Стандарт передачи сообщений (Message Passing Interface) для организации взаимодействия между процессами в распределённых вычислительных системах.
NFS	Протокол сетевой файловой системы (Network File System), обеспечивающий совместное использование файлов между компьютерами в сети.

Продолжение на следующей странице

Таблица 1: Термины и определения (продолжение)

Термин	Определение
OpenMP [4]	Открытый стандарт многопоточного программирования для систем с общей памятью. Позволяет распараллеливать выполнение кода на многоядерных CPU с помощью директив компилятора.
SLURM [8]	Система управления рабочей нагрузкой (Simple Linux Utility for Resource Management) для планирования задач и распределения ресурсов в кластерах высокопроизводительных вычислений.
SSH	Протокол безопасного удалённого доступа (Secure Shell), обеспечивающий зашифрованное соединение и аутентификацию пользователей.

Введение

Современные научные и инженерные задачи характеризуются значительным увеличением вычислительной сложности и объёмов обрабатываемых данных. Эта тенденция создаёт устойчивую потребность в разработке инновационных архитектур и методов организации вычислительных процессов. Традиционные однородные системы, основанные исключительно на центральных процессорах, зачастую не способны обеспечить необходимую производительность и энергоэффективность при решении задач в области моделирования физических процессов, анализа больших данных и искусственного интеллекта. В качестве ответа на эти вызовы получают распространение гетерогенные вычислительные системы, сочетаемые с технологиями виртуализации и операционными системами на основе ядра Linux. Такой комплексный подход позволяет достичь оптимального баланса между высокой производительностью, гибкостью управления ресурсами и масштабируемостью инфраструктуры.

Гетерогенные вычислительные системы представляют собой архитектуру, которая объединяет в рамках единой программно-аппаратной платформы разнородные процессорные устройства: многоядерные центральные процессоры общего назначения (CPU), графические процессоры (GPU), а также специализированные ускорители. Ключевым преимуществом такого подхода является возможность распределения вычислительной нагрузки между типами устройств в соответствии с их специализацией, что позволяет минимизировать узкие места и существенно повысить общую эффективность системы.

Экспоненциальный рост требований к ресурсам со стороны таких областей, как климатическое моделирование, молекулярный дизайн или обучение глубоких нейронных сетей, обуславливает критическую важность внедрения гетерогенных вычислительных систем. Они становятся неотъемлемым элементом современных суперкомпьютерных центров и промышленных кластеров, преодолевая ограничения классических однородных архитектур. Интеграция данной парадигмы с платформами виртуализации и операционными системами семейства Linux формирует универсальную и отказоустойчивую среду для высокопроизводительных вычислений.

Однако развёртывание и эксплуатация гетерогенных вычислительных систем сопряжены с рядом технологических сложностей. К ним относятся повышенные требования к квалификации разработчиков, необходимость глубокого понимания архитектурных особенностей каждого типа ускорителей, а также сложность эффективного распределения задач и синхронизации данных между разнородными компонентами, что создаёт дополнительные вызовы в обеспечении производительности.

В этом контексте среды виртуализации выполняют ключевую роль, предоставляя механизмы для изоляции вычислительных нагрузок, динамического распределения аппаратных ресурсов и миграции задач между физическими узлами. Это особенно важно для эффективного использования разнородного оборудования, так как позволяет прозрачно распределять задания между CPU, GPU и другими ускорителями в соответствии с их вычислительным профилем.

Особый интерес представляет платформа виртуализации Hyper-V [5], которая обеспечивает мощную поддержку гетерогенных вычислений через механизм Discrete Device Assignment (DDA). Данная технология позволяет выполнить прямой проброс физических графических ускорителей и других устройств в виртуальные машины, обеспечивая уровень производительности, близкий к нативному. Интеграция Hyper-V с современными операционными системами и его оптимизация для задач высокопроизводительных вычислений делают его релевантным инструментом для построения виртуальных кластеров.

Доминирование операционных систем на базе ядра Linux в сфере суперкомпьютерных технологий [6] объясняется их открытой архитектурой, исключительной гибкостью настройки, способностью масштабироваться на системы любого уровня и широкой совместимостью с аппаратным обеспечением. Статистика рейтинга TOP500 на январь 2025 года подтверждает, что все представленные в нём системы используют ядро Linux, что подчёркивает его статус отраслевого стандарта.

Актуальность настоящего исследования определяется необходимостью разработки доступной и эффективной виртуальной платформы, которая сочетала бы преимущества гипервизора Нурег-V с вычислительными возможностями гетерогенных систем под управлением Linux. Цель работы заключается в проектировании, развёртывании и анализе производительности виртуального вычислительного кластера на базе Нурег-V с поддержкой аппаратного ускорения GPU, предназначенного для выполнения параллельных приложений, разработанных с использованием стандарта MPI и платформы CUDA.

1 Постановка задачи

Цель работы: разработка и экспериментальная проверка параллельного приложения для анализа финансовых транзакций с использованием гетерогенных вычислительных ресурсов виртуального кластера.

В рамках работы необходимо решить следующую задачу:

- Реализовать программное средство на языке C/C++, выполняющее распределённую обработку набора данных о транзакциях
- Программа должна выполнять фильтрацию исходных данных, оставляя только транзакции с суммой **Amount** ≥ 9000.00
- Сгруппировать отфильтрованные данные по интервалам признака **V1** шириной 0.1 единицы
- Для каждой группы вычислить количество отрицательных значений признака **V7**, математическое ожидание признака **V11** и общую сумму **Amount**
- Выбрать 50 групп с наибольшим математическим ожиданием **V11** среди групп с максимальным количеством отрицательных значений **V7**
- Отсортировать полученные группы по убыванию общей суммы **Amount** и вывести результаты в текстовый файл

Программное средство должно использовать комбинированную модель параллельных вычислений:

- Распределённую обработку между узлами кластера с применением стандарта MPI
- Внутриузловое ускорение на графических процессорах с использованием технологии CUDA
- По желанию допускается использование OpenMP для организации многопоточности на CPU

Приложение должно корректно запускаться как задача в системе управления рабочей нагрузкой SLURM и автоматически задействовать все выделенные вычислительные ресурсы (ядра процессоров, потоки, CUDA-ускорители) на предоставленных узлах.

2 Настройка среды разработки

2.1 Конфигурация Hyper-V

2.1.1 Установка и подготовка Hyper-V

Для создания виртуального кластера с поддержкой гетерогенных вычислений необходимо обеспечить корректную установку и настройку гипервизора Hyper-V. В рамках настоящей работы программная среда Hyper-V уже развёрнута на вычислительных станциях лаборатории кафедры. Однако для воспроизведения инфраструктуры на других компьютерах требуется соблюдение определённых технических требований и последовательное выполнение этапов подготовки системы.

Ключевым требованием к оборудованию является поддержка технологий виртуализации на уровне процессора (Intel VT-x или AMD-V) и наличие в BIOS/UEFI активированной опции виртуализации. Дополнительно требуется 64-разрядная версия Windows 10/11 Pro, Enterprise или Education, поскольку версии Home Edition не поддерживают роль Hyper-V. Минимальные требования к ресурсам хост-системы включают 4 ГБ оперативной памяти (рекомендуется 8 ГБ и более) и свободное дисковое пространство объёмом не менее 20 ГБ для размещения виртуальных машин.

Процесс активации гипервизора в операционной системе Windows осуществляется через компоненты операционной системы. Для этого необходимо открыть раздел «Параметры Windows», перейти в категорию «Приложения», выбрать пункт «Программы и компоненты» и активировать функцию «Включение или отключение компонентов Windows». В появившемся диалоговом окне требуется установить флажок напротив пункта «Hyper-V», включив при этом дочерние компоненты: «Платформа Hyper-V» для основных функций виртуализации и «Средства управления Hyper-V» для администрирования инфраструктуры. После подтверждения выбора система инициирует установку необходимых компонентов и потребует перезагрузки компьютера для завершения процесса.

По завершении перезагрузки в операционной системе становится доступен основной инструмент администрирования — «Диспетчер Hyper-V», предоставляющий графический интерфейс для управления виртуальными машинами, виртуальными коммутаторами и другими ресурсами гипервизора. Дополнительно проверяется работоспособность служб виртуализации через консоль управления PowerShell с использованием командлета Get-VMHost, который отображает текущее состояние гипервизора и доступные вычислительные ресурсы.

2.1.2 Конфигурация основных параметров Hyper-V

При настройке Hyper-V [5] были определены ключевые параметры для обеспечения производительности и удобства эксплуатации. В настройках Hyper-V путь хранения виртуальных машин и их дисков изменён на D:\VMs\.

Архитектура процессора включает 8 физических ядер и 16 логических потоков благодаря технологии гиперпоточности. Параметр NUMA spanning оставлен активным, что позволяет виртуальным машинам использовать память из разных NUMA-узлов, упрощая управление ресурсами. При этом предусмотрена возможность отключения данной функции для сценариев, требующих строгой привязки памяти и максимальной производительности.

Функция миграции виртуальных машин обеспечивает их перемещение между хостами с минимальным временем простоя, однако создаёт повышенную нагрузку на сетевые и дисковые ресурсы. Для поддержания стабильности системы количество одновременных миграций ограничено значением по умолчанию — двумя параллельными операциями.

Для сетевой конфигурации изначально применялся стандартный коммутатор типа Default Switch, обеспечивающий базовое сетевое подключение.

2.2 Настройка сети

Целью конфигурации являлось создание изолированной внутренней сети для виртуальных машин с организацией NAT-доступа во внешнюю сеть через хост-систему. Для реализации задачи в Hyper-V Manager был создан внутренний виртуальный коммутатор с именем **vmNat** через раздел Virtual Switch Manager, доступный в правой панели интерфейса после выбора локального сервера в дереве управления.

После создания коммутатора в операционной системе хоста появился виртуальный сетевой адаптер **vEthernet (vmNat)**. Для его настройки использовался PowerShell с правами администратора. Сначала через команду **ipconfig** выполнялась идентификация целевого сетевого интерфейса, после чего назначался статический IP-адрес шлюза. В качестве адреса маршрутизатора применялся стандартный шаблон 10.200.172.200 (где 172 — уникальный номер подсети, сформированный последним октетом IP-адреса машины). Привязка адреса осуществлялась командой:

```
1 New-NetIPAddress -InterfaceAlias "vEthernet (vmNat)" `
2 -IPAddress 10.200.172.200 -PrefixLength 24
```

До выполнения операции удалялись конфликтующие конфигурации с помощью **Remove-NetIPAddress**, а текущие настройки проверялись через **Get-NetIPAddress**. Пример корректной конфигурации виртуального адаптера:

```
1 IPAddress           : 10.200.172.200
2 InterfaceIndex      : 4
3 InterfaceAlias       : vEthernet (vmNat)
4 AddressFamily        : IPv4
5 Type                 : Unicast
6 PrefixLength         : 24
7 PrefixOrigin         : Manual
8 SuffixOrigin         : Manual
9 AddressState         : Preferred
```

Завершающим этапом стала настройка NAT-трансляции для обеспечения интернет-доступа виртуальных машин. Перед созданием нового правила проводилась очистка существующих конфигураций через **Remove-NetNat**, а актуальные параметры проверялись командами **Get-NetNat** и **Get-NetNatStaticMapping**. Центральной операцией стал запуск команды:

```
1 New-NetNat -Name "vmNat" `
2 -InternalIPInterfaceAddressPrefix 10.200.172.0/24
```

Успешное выполнение конфигурации подтверждается выводом системы. Для проверки состояния NAT используется команда **Get-NetNat**, демонстрирующая активное правило с параметрами трансляции:

```
1 Name                  : vmNat
2 InternalIPInterfaceAddressPrefix : 10.200.172.0/24
3 IcmpQueryTimeout      : 30
4 TcpEstablishedConnectionTimeout : 1800
5 TcpTransientConnectionTimeout   : 120
6 UdpIdleSessionTimeout  : 120
7 Store                  : Local
8 Active                 : True
```

2.3 Настройка виртуальной машины

Для создания виртуального узла с последующим включением в кластер были выполнены следующие шаги:

- Открыт Диспетчер Нурег-V.
- Запущен мастер создания виртуальной машины через пункт меню Создать → Виртуальная машина.
- На этапе указания имени и расположения заданы параметры:
 - Имя виртуальной машины: **ayzekcpu1**
 - Путь хранения: **D:\VMs** (каталог по умолчанию для виртуализации)
- Выбрано поколение виртуальной машины:
 - Установлено Поколение 2 как обязательная конфигурация для корректной работы с GPU и CUDA
 - Обоснование выбора:
 - * Поддержка современных технологий виртуализации
 - * Оптимизация использования аппаратных ресурсов
 - * Гарантированная совместимость с драйверами GPU
 - * Повышение производительности вычислений
- Настроены параметры оперативной памяти:
 - Выделено 4096 МБ оперативной памяти
 - Отключена опция «Использовать динамическую память» из-за её несовместимости с CUDA
- Выполнена базовая сетевая конфигурация:
 - Выбран коммутатор **DefaultSwitch** как временный вариант подключения
 - Запланирована последующая замена на специализированный коммутатор **vmNat**
- Настроено виртуальное хранилище:
 - Имя файла виртуального диска: **ayzekcpu1.vhdx**
 - Путь размещения: **D:\VMs**
 - Объём диска: 127 ГБ с динамическим распределением физического пространства
- Определён порядок установки операционной системы: выбран вариант отложенной установки ОС после завершения создания виртуальной машины
- Проведена финальная проверка конфигурации:
 - Подтверждены все параметры на этапе сводного обзора
 - Завершён процесс создания нажатием кнопки «Готово»

2.4 Настройка параметров виртуальной машины

Для завершения конфигурации виртуальной машины были выполнены следующие операции в диспетчере Нурег-V:

- Открыты параметры виртуальной машины **ayzekcpu1** через контекстное меню
- Настроены параметры процессора:
 - Установлено количество виртуальных процессоров: 2
 - Параметры распределения ресурсов оставлены по умолчанию:

- * Резерв для виртуальных машин (%)
- * Процент от общего объема системных ресурсов (%)
- Задан относительный вес для приоритезации доступа к процессорным ресурсам при конкуренции между виртуальными машинами
- Активирована поддержка NUMA:
 - Включена опция «Использовать аппаратную топологию NUMA»
 - Параметры максимального числа процессоров и объема памяти для одного узла оставлены без изменений
- Сконфигурирован SCSI-контроллер: подключен DVD-дисковод для работы с ISO-образом операционной системы
- Настроен DVD-дисковод:
 - Выбран тип подключения «Файл образа»
 - Указан путь к ISO-образу: `D:\ubuntu.iso`
- Изменены параметры безопасности: отключена опция «Включить безопасную загрузку» из-за несовместимости с кастомным ядром Linux и DKMS
- Отключены контрольные точки: функция деактивирована для предотвращения конфликтов с GPU и CUDA-приложениями
- Настроены автоматические действия:
 - При запуске хоста: «Ничего» (запрет автоматической загрузки VM)
 - При завершении работы: «Завершить работу ОС на виртуальной машине» для сохранения целостности данных
- Подтверждено отключение динамической памяти для обеспечения совместимости с GPU
- Настроена последовательность загрузки во встроенном ПО:
 - Первое устройство: `ayzekcpu1.vhdx` (жёсткий диск)
 - Второе устройство: сетевой адаптер `Default Switch`
 - Третье устройство: DVD-дисковод с операционной системой
- Нажата кнопка «Применить» для сохранения всех настроек

Все параметры соответствуют требованиям для последующей установки операционной системы и интеграции узла в кластер.

2.5 Установка операционной системы

Установка операционной системы выполнялась на виртуальную машину `ayzekcpu1` через консоль Hyper-V после запуска машины двойным щелчком и нажатия кнопки «Пуск». На этапе начальной настройки Ubuntu Server выбран английский язык системы и раскладка клавиатуры English (US). Для дисковой подсистемы создавались два раздела: раздел подкачки объёмом 50 МБ и основной раздел размером 4 ГБ с файловой системой `ext4`.

После завершения базовой установки выполнены критические послеустановочные операции. В консоли системы обновлён пароль учётной записи `root` через последовательность команд:

```
1 sudo su -
2 passwd
```

Проведено полное обновление пакетов:

```
1 sudo apt update
2 sudo apt upgrade -y
```

Установлено программное обеспечение для кластерных вычислений и администрирования:

- `openmpi-bin openmpi-doc libopenmpi-dev` — библиотеки и инструменты MPI
- `slurmd` — демон системы управления заданиями Slurm
- `iputils-ping build-essential nano` — утилиты для сетевой диагностики, компиляции программ и редактирования конфигураций

Для корректной работы сетевой конфигурации отключён автоматический менеджер `cloud-init`. Создан файл `/etc/cloud/cloud.cfg.d/99-disable-network-config.cfg` со следующим содержанием:

```
1 network: {config: disabled}
```

Это действие предотвращает конфликты при ручной настройке сети через `netplan`.

В файле `/etc/hosts` добавлены записи для будущих узлов кластера в соответствии с сетевой схемой:

```
1 10.200.172.172 ayzekcpu1 ayzekcpu1.hpc
2 10.200.172.173 ayzekcpu2 ayzekcpu2.hpc
3 10.200.172.174 ayzekgpu1 ayzekgpu1.hpc
4 10.200.172.175 ayzekgpu2 ayzekgpu2.hpc
```

Такая конфигурация обеспечивает разрешение имён узлов без обращения к внешним DNS-серверам.

Завершающим этапом проведена очистка кэша пакетов:

```
1 sudo apt clean
```

Данная операция освобождает дисковое пространство и минимизирует риски, связанные с использованием устаревших пакетов. Все настройки выполнены с учётом требований совместимости с CUDA-стеком и последующей интеграции в распределённую вычислительную среду.

2.6 Клонирование виртуальных машин

Для быстрого развёртывания однотипных узлов кластера использовался механизм экспорта и импорта виртуальных машин Nureg-V. Процедура выполнялась в следующей последовательности:

Исходная эталонная виртуальная машина `ayzekcpu1` была принудительно выключена через диспетчер Nureg-V для обеспечения целостности данных. Через контекстное меню машины запущена операция **Export...** с указанием временной директории `C:\tmp\export`. После завершения экспорта исходная ВМ была переименована в `ayzekcpu1a`, а соответствующий файл виртуального диска `ayzekcpu1.vhdx` изменён на `ayzekcpu1a.vhdx` в каталоге `D:\VMs`. Данная операция необходима для разделения хранилищ исходной и клонируемой систем.

Импорт копии выполнен через пункт меню **Import Virtual Machine...** с выбором режима «Копировать виртуальную машину (создать новую уникальную идентификацию)». В процессе импорта новой машине присвоено имя `ayzekcpu2` с сохранением всех настроек аппаратной конфигурации эталона.

2.6.1 Запуск служб

На рабочих узлах перезапущена служба **slurmd**:

```
1 sudo systemctl restart slurmd
```

На главном узле активирована служба **slurmctld**:

```
1 sudo systemctl restart slurmctld
```

Проверено состояние служб:

```
1 systemctl status slurmd
2 systemctl status slurmctld
```

Убедился, что все файлы и директории, указанные в **slurm.conf**, существуют и принадлежат пользователю **slurm**:

```
1 chown -R slurm:slurm /var/spool/slurm*
```

Проверена видимость узлов кластера:

```
1 sinfo
2 scontrol show nodes
```

2.7 Настройка серверной части (NFS и CUDA)

2.7.1 NFS-шеринг

На главном узле кластера установлена NFS-серверная компонента:

```
1 sudo apt install -y nfs-kernel-server
```

На всех рабочих узлах выполнена установка клиентских пакетов:

```
1 sudo apt install -y nfs-common
```

На всех узлах создан общий каталог:

```
1 sudo mkdir -p /mnt/share
```

В файл **/etc/exports** главного узла добавлена строка экспорта:

```
1 /mnt/share *(rw,sync,no\_subtree\_check)
```

Настройки применены и проверен статус службы:

```
1 sudo exportfs -ra
2 sudo systemctl status nfs-server
```

На клиентских узлах для автоматического монтирования при загрузке в файл **/etc/fstab** добавлена запись:

```
1 ayzekcpu1:/mnt/share /mnt/share nfs defaults 0 0
```

Конфигурация применена командой:

```
1 sudo mount -a
```

После настройки проверена работоспособность: файл, созданный в **/mnt/share** на главном узле, отображается на всех клиентских машинах без задержек.

2.7.2 Проверка пакетов CUDA

На основном узле выполнена проверка наличия и версии компилятора CUDA [3]:

```
1 nvcc --version
```

При обнаружении устаревшей версии или отсутствия пакета выполнено удаление старого дистрибутивного пакета:

```
1 sudo apt remove --purge -y nvidia-cuda-toolkit
```

Добавлен официальный репозиторий NVIDIA для Ubuntu 22.04:

```
1 wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/
  x86_64/cuda-keyring_1.1-1_all.deb
2 sudo dpkg -i cuda-keyring_1.1-1_all.deb
3 sudo apt update
```

Установлена актуальная версия CUDA Toolkit 12.8:

```
1 sudo apt install -y cuda-toolkit-12-8
```

В файл `/.bashrc` добавлены переменные окружения:

```
1 echo 'export PATH=/usr/local/cuda-12.8/bin:$PATH' >> ~/.bashrc
2 echo 'export LD_LIBRARY_PATH=/usr/local/cuda-12.8/lib64:$LD_LIBRARY_PATH'
  ' >> ~/.bashrc
3 source ~/.bashrc
```

Финальная проверка версии компилятора подтвердила корректность установки:

```
1 nvcc --version
```

Теперь тестовый код, собранный на главном узле, может быть запущен на машинах через Slurm без дополнительных настроек окружения.

3 Проверка работоспособности кластера

3.1 Проверка работоспособности CUDA

Для верификации корректности настройки CUDA-окружения выполнена тестовая программа, выводящая детальную информацию о доступных вычислительных ресурсах. Программа отображает сведения об установленных GPU-устройствах, включая их количество, модель, объём видеопамати и количество CUDA-ядер.

Тестовый вывод содержит следующие диагностические данные:

- Информацию о каждом GPU-устройстве (название, вычислительная возможность, тактовая частота)
- Количество мультипроцессоров и CUDA-ядер на устройстве
- Размеры блоков и сетки для оптимального распределения вычислений
- Объём доступной глобальной, разделяемой и постоянной памяти
- Результаты тестовых вычислений с указанием времени выполнения
- Статус выполнения операций на GPU и коды возврата

При успешном выполнении программа демонстрирует корректную работу всех компонентов CUDA-стека, включая выделение памяти, передачу данных между хостом и устройством, запуск ядер и получение результатов. Диагностическое сообщение подтверждает доступность всех вычислительных ресурсов GPU для приложений.

Тестирование проводилось на узлах `ayzekgpu1` и `ayzekgpu2` с использованием Slurm для распределения задач. Все операции завершились без ошибок.

3.2 Проверка MPI

Для верификации работы распределённых вычислений в кластере была разработана тестовая программа на основе MPI. Программа инициализирует MPI-окружение, определяет количество процессов и их ранги, получает информацию об именах вычислительных узлов, а затем выводит диагностические сообщения для каждого процесса. Исходный код программы содержит стандартные MPI-функции: `MPI_Init`, `MPI_Comm_size`, `MPI_Comm_rank`, `MPI_Get_processor_name` и `MPI_Finalize` [2].

Компиляция тестовой программы выполнена на главном узле с использованием MPI-компилятора:

```
1 mpicxx mpi\_test.cpp -o mpi\_test
```

Исполняемый файл перемещён в общий каталог NFS-шары:

```
1 mv mpi\_test /mnt/share/
```

Для запуска задачи на кластере создан Slurm-скрипт `mpi_test.slurm` в каталоге `/mnt/share` со следующими параметрами:

- Имя задания: `mpi_test`
- Раздел кластера: `hpc_part`
- Список узлов: `ayzekcpu1,ayzekcpu2,ayzekgpu1,ayzekgpu2`
- Количество задач на узел: 2
- Файлы вывода и ошибок: `mpi_test.out, mpi_test.err`

При создании скрипта в Windows выполнена конвертация формата переносов строк:

```
1 dos2unix mpi\_test.slurm
```

Задача отправлена в очередь кластера:

```
1 sbatch mpi\_test.slurm
```

Результаты выполнения просмотрены в файле вывода:

```
1 cat mpi\_test.out
```

Вывод программы подтвердил корректную работу MPI-окружения: 8 процессов распределены по 4 узлам кластера (по 2 процесса на узле). Каждый процесс вывел свой ранг, общее количество процессов и имя узла, на котором он выполнялся. Полный исходный код программы `mpi_test.cpp` приведен в Приложении Б.

4 Анализ поставленной задачи и подходы к решению

4.1 Постановка задачи анализа данных

В рамках работы требуется разработать параллельное приложение для сложного аналитического процессинга финансовых транзакций с использованием гетерогенных вычислительных ресурсов. Алгоритм обработки данных включает следующие этапы:

1. **Фильтрация данных:** загрузить датасет кредитных транзакций и отфильтровать записи, оставив только транзакции с суммой (`Amount`) ≥ 9000.00 .
2. **Группировка:** сгруппировать отфильтрованные данные по интервалам признака `V1` с шагом 0.1 единицы.
3. **Вычисление статистик:** для каждой группы вычислить:
 - Количество отрицательных значений признака `V7`
 - Математическое ожидание признака `V11`
 - Общую сумму `Amount` всех транзакций в группе
4. **Селекция групп:** выбрать 50 групп, которые имеют наибольшее математическое ожидание `V11` среди тех групп, где количество отрицательных значений `V7` является максимальным.
5. **Сортировка и вывод:** отсортировать полученные 50 групп по убыванию их общей суммы `Amount` и вывести результаты в текстовый файл.

Объём исходных данных превышает 550,000 записей, что требует эффективного использования распределённых вычислений и аппаратного ускорения.

4.2 Неоптимальные подходы к решению

Рассмотрим несколько подходов, которые имеют существенные недостатки в контексте поставленной задачи.

4.2.1 Подход 1: Синхронная модель с последовательными барьерами

Идея: Реализовать строгую синхронную модель, где каждый этап обработки требует полной синхронизации всех процессов MPI. После чтения данных устанавливается барьер, после фильтрации — следующий барьер, после группировки — третий, и т.д. Все процессы должны завершить каждый этап, прежде чем система перейдёт к следующему.

Недостатки:

- **Простой быстрых узлов:** GPU-узлы, способные обрабатывать данные значительно быстрее CPU-узлов, вынуждены простаивать в ожидании завершения работы медленных CPU-узлов на каждом этапе.
- **Накладные расходы на синхронизацию:** Частые вызовы `MPI_Barrier` создают значительные накладные расходы, особенно при большом количестве процессов.
- **Отсутствие конвейерной обработки:** Невозможность начать следующий этап обработки до полного завершения предыдущего всеми процессами.
- **Чувствительность к дисбалансу нагрузки:** Любая неравномерность в распределении данных или производительности узлов приводит к простоям всей системы.

4.2.2 Подход 2: Статическое распределение данных без учёта производительности GPU

Идея: Равномерно распределить данные между всеми узлами кластера, игнорируя различия в производительности между CPU и GPU. Каждый узел получает одинаковый объём данных для обработки, независимо от своих аппаратных возможностей.

Недостатки:

- **Неэффективное использование GPU:** GPU-узлы, обладающие значительно большей вычислительной мощностью, получают тот же объём работы, что и CPU-узлы. В результате они быстро завершают обработку своей порции данных и простаивают.
- **Дисбаланс времени выполнения:** Общее время выполнения определяется самым медленным CPU-узлом, в то время как GPU-узлы остаются недозагруженными.
- **Отсутствие адаптивности:** Система не может динамически перераспределять нагрузку в зависимости от фактической производительности каждого узла.
- **Невозможность компенсации аппаратных различий:** Различия в производительности между разными моделями GPU или CPU не учитываются.

4.2.3 Подход 3: Блокирующая отправка результатов с ожиданием главного узла

Идея: Использовать блокирующие операции MPI (MPI_Send) для отправки результатов с рабочих узлов на главный. Главный узел последовательно ожидает данные от каждого процесса в строгом порядке (Rank 1, затем Rank 2, и т.д.), обрабатывая их только после полного получения.

Недостатки:

- **Последовательная обработка на главном узле:** Главный узел не может начать обработку данных от Rank 2, пока полностью не обработает данные от Rank 1, даже если данные от Rank 2 уже получены.
- **Блокировка отправляющих процессов:** Рабочие узлы блокируются при отправке данных, не могут выполнять другие задачи в ожидании подтверждения получения.
- **Невозможность перекрытия вычислений и коммуникаций:** Вычисления на главном узле и передача данных с рабочих узлов не могут выполняться параллельно.
- **Зависимость от порядка завершения:** Если какой-то процесс завершил вычисления позже других, он задерживает обработку всех результатов на главном узле.

4.3 Финальное решение: Гибридная параллельная архитектура

Предлагаемое решение реализует эффективную архитектуру, преодолевающую недостатки рассмотренных выше подходов. Основные принципы:

4.3.1 Подход 3: Последовательная обработка с барьерами синхронизации

Идея: Использовать строгую синхронизацию между этапами обработки: все узлы должны завершить чтение данных, прежде чем начать фильтрацию; завершить фильтрацию, прежде чем начать группировку, и т.д.

Недостатки:

- **Низкая эффективность:** Быстрые узлы простаивают в ожидании медленных на каждом этапе обработки.

- **Накладные расходы на синхронизацию:** Частые барьеры MPI существенно снижают общую производительность.
- **Отсутствие конвейерной обработки:** Невозможность начать следующий этап до полного завершения предыдущего.
- **Сложность отладки:** Любая задержка на одном узле влияет на производительность всей системы.

4.4 Финальное решение: Гибридная параллельная архитектура

Предлагаемое решение реализует эффективную архитектуру, преодолевающую недостатки рассмотренных выше подходов. Основные принципы:

4.4.1 Динамическое распределение нагрузки

Идея: Реализована система взвешенного распределения данных, учитывающая аппаратные характеристики узлов. GPU-узлы получают больше данных для обработки, чем CPU-узлы, что компенсирует их более высокую производительность. Веса распределения настраиваются через переменные окружения, позволяя адаптироваться к конкретной конфигурации кластера.

Преимущества:

- **Балансировка нагрузки:** GPU-узлы получают больший объём данных, что минимизирует их простой.
- **Автоматическая адаптация:** Система определяет наличие GPU на узлах и соответствующим образом распределяет нагрузку.
- **Гибкость:** Веса могут быть настроены для оптимального соотношения в конкретном кластере.

4.4.2 Гибридная обработка: MPI + CUDA

Идея: Использование комбинированной модели параллельных вычислений:

- **Межузловая параллелизация:** Распределение данных между узлами кластера с использованием стандарта MPI.
- **Внутриузловое ускорение:** Использование технологии CUDA для ускорения вычислений на GPU-узлах.
- **Автоматическое переключение:** При отсутствии доступных GPU система автоматически переключается на CPU-реализацию алгоритма.

Преимущества:

- **Максимальное использование аппаратуры:** Одновременное использование CPU и GPU ресурсов.
- **Отказоустойчивость:** Автоматический fallback на CPU при проблемах с GPU.
- **Прозрачность для пользователя:** Единый интерфейс программы независимо от используемого оборудования.

4.4.3 Асинхронная агрегация результатов

Идея: Главный узел (Rank 0) начинает обработку результатов сразу после их получения от любого готового процесса, не дожидаясь завершения всех остальных. Процессы используют

неблокирующую отправку данных (`MPI_Isend`), что позволяет им продолжать работу без ожидания подтверждения.

Преимущества:

- **Минимизация времени простоя:** Главный узел начинает обработку данных сразу, не дожидаясь медленных процессов.
- **Устранение блокировок:** Процессы не блокируются при отправке результатов.
- **Конвейеризация:** Перекрытие вычислений на рабочих узлах с обработкой результатов на главном узле.

4.4.4 Оптимизированная GPU-реализация

Идея: Для GPU-обработки реализован специализированный алгоритм агрегации, который включает:

- **Вычисление идентификаторов периодов** для каждой записи (на основе значения `V1`).
- **Сортировку записей** по идентификаторам периодов с использованием `counting sort` для небольших диапазонов или `quicksort` для больших.
- **Группировку отсортированных записей** и вычисление статистик для каждого периода.
- **Динамический выбор ядра:** Использование разных вычислительных ядер в зависимости от количества периодов и размера данных.

Преимущества:

- **Эффективное использование памяти GPU:** Минимизация копирований данных между хостом и устройством.
- **Адаптация к характеристикам данных:** Автоматический выбор оптимального алгоритма сортировки.
- **Подробное логирование:** Детальная информация о времени выполнения каждого этапа GPU-обработки.

4.4.5 Этапы работы финального решения

1. **Инициализация:** Все процессы MPI определяют свои ранги и размер коммунитатора. Проверяется доступность GPU и вычисляются веса распределения.
2. **Параллельное чтение:** Каждый процесс читает свою часть файла данных, определяемую на основе весов распределения. Фильтрация по `Amount ≥ 9000.00` выполняется во время чтения.
3. **Локальная агрегация:** Каждый процесс агрегирует свои данные по периодам `V1`. GPU-процессы используют CUDA-ускорение, CPU-процессы — оптимизированную CPU-реализацию.
4. **Асинхронная отправка:** Все процессы (кроме Rank 0) отправляют свои результаты на главный узел с использованием неблокирующих операций MPI и немедленно продолжают работу.
5. **Постепенная агрегация:** Rank 0 принимает результаты от процессов в произвольном порядке, сразу начиная их обработку. Объединяет статистики по одинаковым периодам от разных процессов.

6. **Финальный отбор:** Rank 0 выполняет логику отбора: находит группы с максимальным количеством отрицательных `V7`, среди них выбирает 50 с наибольшим математическим ожиданием `V11`, сортирует по убыванию суммы `Amount`.
7. **Запись результата:** Rank 0 сохраняет финальный набор из 50 групп в CSV-файл.

4.5 Пояснение выбора решения

Ключевыми преимуществами выбранного решения являются:

- **Эффективное использование ресурсов:** Динамическое распределение нагрузки в соответствии с аппаратными возможностями узлов.
- **Минимизация простоя:** Асинхронная модель обмена данными и обработки результатов.
- **Гибкость:** Поддержка как GPU, так и CPU вычислений с автоматическим переключением.
- **Масштабируемость:** Возможность обработки датасетов значительно большего размера за счёт распределённой архитектуры.

Данная реализация успешно решает поставленную задачу анализа финансовых транзакций, обеспечивая высокую производительность и эффективное использование вычислительных ресурсов виртуального кластера.

5 Анализ датасета транзакций

5.1 Общее описание датасета

Данный датасет содержит информацию о транзакциях по кредитным картам, совершённых европейскими держателями карт в течение 2023 года. Основные характеристики набора данных:

- **Объём данных:** 568,632 записи (транзакции)
- **Период данных:** 2023 год
- **География:** Европейские страны
- **Анонимизация:** Все данные были анонимизированы для защиты личности держателей карт
- **Цель создания:** Разработка и тестирование алгоритмов обнаружения мошеннических операций

Датасет содержит как легитимные (нормальные), так и мошеннические транзакции, что позволяет обучать модели бинарной классификации. Классический дисбаланс классов характерен для подобных задач обнаружения мошенничества.

5.2 Структура и описание колонок

Датасет содержит 31 колонку следующей структуры:

- **id** - Уникальный идентификатор транзакции
 - Тип: целочисленный
 - Назначение: однозначная идентификация каждой записи
 - Особенность: последовательная нумерация, не несёт смысловой нагрузки для анализа
- **V1, V2, ..., V28** - Анонимизированные признаки транзакции
 - Тип: вещественные числа (float)
 - Количество: 28 признаков
 - Происхождение: результат применения методов снижения размерности (PCA) к исходным данным
 - Особенности:
 - * Исходные признаки были преобразованы для защиты конфиденциальности
 - * Признаки содержат информацию о различных аспектах транзакции: время, местоположение, тип операции, данные о держателе карты и т.д.
 - * Названия V1 – V28 не раскрывают семантику признаков из-за требований конфиденциальности
 - * Признаки имеют разные масштабы и распределения
- **Amount** - Сумма транзакции
 - Тип: вещественное число (float)
 - Единицы измерения: предположительно евро
 - Формат: два десятичных знака (например, 17982.10)
 - Особенности:

- * В задании используется для фильтрации: $\text{Amount} \leq 9000.00$
- * Диапазон значений может существенно варьироваться
- * Высокие суммы могут быть более интересны для анализа мошенничества
- **Class** - Метка класса (целевая переменная)
 - Тип: бинарный целочисленный
 - Значения:
 - * 0 - Легитимная (нормальная) транзакция
 - * 1 - Мошенническая транзакция
 - Особенности:
 - * Сильный дисбаланс классов (мошеннических транзакций значительно меньше)
 - * Используется для обучения и оценки моделей обнаружения мошенничества
 - * В аналитической задаче может использоваться для верификации результатов

5.3 Пример записи датасета

Рассмотрим пример строки из датасета для лучшего понимания структуры:

```
id,V1,V2,V3,V4,V5,V6,V7,V8,V9,V10,V11,V12,V13,V14,V15,V16,V17,V18,V19,V20,V21,V22,V23,V24,V25,
0,-0.26064780489439815,-0.46964845005363426,2.4962660826315637,-0.08372391267814633,
0.1296812361545678,0.7328982498449426,0.5190136179018007,-0.13000604758867731,
0.7271592691096374,0.6377345411881967,-0.9870200098799841,0.2934381004820159,
-0.9413861250929441,0.5490198936308889,1.8048785784684263,0.2155979938725388,
0.5123066605849524,0.3336437173298195,0.12427015635408474,0.09120189881650709,
-0.11055167961012961,0.21760614382950005,-0.1347944948772723,0.1659591154312752,
0.1262799761446219,-0.43482398075374323,-0.08123010860166756,-0.1510454864555771,
17982.1,0
```

Интерпретация примера:

- **ID:** 0 - первая транзакция в датасете
- **$V_1 - V_{28}$:** Значения нормализованных признаков, например:
 - $V_1 = -0.2606$ (отрицательное значение)
 - $V_3 = 2.4963$ (положительное значение, значительно выше среднего)
 - $V_7 = 0.5190$ (положительное значение, используется в задании)
 - $V_{11} = -0.9870$ (отрицательное значение, используется в задании)
- **Amount:** 17982.10 - крупная транзакция (удовлетворяет условию фильтрации)
- **Class:** 0 - легитимная транзакция (не мошенническая)

5.4 Особенности обработки для поставленной задачи

Для выполнения поставленной задачи анализа требуются следующие преобразования данных:

- **Фильтрация:** Отбор только транзакций с суммой ≥ 9000.00 (колонок Amount)
- **Группировка:** Агрегация данных по интервалам признака V_1 с шагом 0.1

- **Анализ:** Работа с признаками:
 - V_7 - определение отрицательных значений
 - V_11 - вычисление математического ожидания
 - Amount - расчёт общей суммы в группе
- **Селекция:** Выбор 50 групп по комбинированным критериям

Объём датасета (более 568 тысяч записей) требует эффективной обработки с использованием параллельных вычислений, особенно с учётом необходимости фильтрации, группировки и сложных статистических вычислений.

6 Разработка программного средства

6.1 Архитектура программного средства

Разработанное программное средство реализует гибридную модель параллельных вычислений, сочетающую распределённую обработку данных между узлами виртуального кластера с использованием стандарта MPI и внутриузловое ускорение вычислений на графических процессорах с применением технологии CUDA. Дополнительно предусмотрен резервный путь выполнения на CPU в случае отсутствия доступных GPU-устройств.

Обработка финансовых транзакций организована по конвейерному принципу и включает следующие логические этапы:

- динамическое распределение вычислительных ресурсов между MPI-процессами;
- параллельное чтение входного CSV-файла с перекрытием границ;
- фильтрацию записей по условию $\text{Amount} \geq 9000.00$;
- агрегацию данных по интервалам признака $V1$ шириной 0.1;
- вычисление агрегированных статистик для каждого интервала;
- распределённый сбор и редукцию результатов;
- глобальный отбор и сортировку итоговых групп;
- запись результатов в выходной файл.

Каждый MPI-процесс выполняет полный цикл обработки своей доли данных, что минимизирует объём межпроцессного взаимодействия и обеспечивает хорошую масштабируемость приложения на гетерогенных вычислительных системах.

6.2 Описание структур данных

Для представления входных данных и промежуточных результатов используются несколько специализированных структур.

Структура **Record** описывает одну финансовую транзакцию и содержит только те признаки, которые участвуют в вычислениях:

- $V1$ — признак для группировки;
- $V7$ — признак для подсчёта отрицательных значений;
- $V11$ — признак для вычисления математического ожидания;
- **Amount** — сумма транзакции.

Структура **PeriodStats** используется на этапе локальной агрегации и аккумулирует статистику по одному интервалу признака $V1$. Индекс интервала вычисляется как $\lfloor V1/0.1 \rfloor$.

На финальном этапе используется структура **Interval**, которая хранит уже нормализованные значения (математическое ожидание $V11$ вместо суммы), а также итоговые агрегаты, готовые к глобальной сортировке и выводу.

6.3 Динамическое распределение вычислительных ресурсов

Ключевой особенностью реализации является динамическое распределение объёма обрабатываемых данных между MPI-процессами на основе их аппаратных возможностей.

Каждый процесс определяет наличие доступного GPU-устройства. Эта информация собирается с использованием коллективной операции `MPI_Allgather`. На основе полученного вектора флагов формируется массив весов процессов:

- CPU-процессы получают базовый вес w_{cpu} ;
- GPU-процессы получают увеличенный вес w_{gpu} , отражающий их более высокую пропускную способность.

Весовые коэффициенты могут быть заданы через переменные окружения `CPU_PROCESS_WEIGHT` и `GPU_PROCESS_WEIGHT`, что позволяет адаптировать приложение к конкретной конфигурации кластера без изменения кода.

Далее веса нормализуются и преобразуются в целочисленные доли, определяющие объём данных для каждого MPI-процесса. Эти доли используются при вычислении диапазонов байт входного файла, подлежащих чтению каждым процессом.

6.4 Параллельное чтение входных данных

Чтение входного CSV-файла выполняется параллельно всеми MPI-процессами. Для этого файл логически разбивается на непересекающиеся диапазоны байт, пропорциональные вычисленным долям нагрузки.

Для корректной обработки записей, пересекающих границы диапазонов, используется механизм перекрытия (`overlap`), при котором каждый процесс читает небольшой дополнительный участок данных за пределами своего основного диапазона.

После чтения выполняется разбор строк CSV и фильтрация записей по условию `Amount ≥ 9000.00`. Таким образом, уже на этапе ввода исключаются нерелевантные данные, что снижает объём последующих вычислений и межпроцессного обмена.

6.5 Локальная агрегация и GPU-ускорение

Каждый MPI-процесс независимо агрегирует свои записи по интервалам признака `V1`. Для каждого интервала вычисляются:

- минимальное и максимальное значение `V1`;
- сумма `V11`;
- количество отрицательных значений `V7`;
- сумма `Amount`;
- общее количество записей.

При наличии доступного GPU используется CUDA-реализация агрегации. Данные предварительно преобразуются в компактный формат и передаются на устройство. Каждый CUDA-поток обрабатывает одну запись, выполняя атомарное обновление агрегатов соответствующего интервала.

В случае недоступности GPU автоматически используется CPU-реализация, что обеспечивает корректную работу приложения на любых узлах кластера.

6.6 Сбор и редукция интервалов через MPI

После локальной агрегации статистики преобразуются в интервалы, содержащие математическое ожидание `V11`. Далее выполняется распределённый сбор данных на процессе с рангом 0.

Для минимизации времени ожидания используется асинхронная схема передачи: процессы с рангом больше нуля отправляют свои данные с помощью `MPI_Isend` и немедленно продолжают выполнение, не блокируясь на операции передачи.

Процесс с рангом 0 принимает данные от любых готовых процессов с использованием `MPI_ANY_SOURCE`. Полученные интервалы немедленно добавляются в общий массив, а после завершения приёма выполняется объединение интервалов с одинаковым индексом периода.

Такая схема позволяет эффективно перекрывать вычисления и коммуникации, что особенно важно при неоднородной производительности GPU- и CPU-узлов.

6.7 Финальная обработка и вывод результатов

После объединения интервалов на процессе с рангом 0 выполняется глобальный отбор результатов в соответствии с условиями задачи.

Сначала интервалы сортируются по убыванию количества отрицательных значений $V7$. Определяется максимальное значение этого показателя, и отбираются все интервалы, ему соответствующие. Если таких интервалов менее заданного числа N , выборка расширяется до первых N интервалов.

Далее выполняется сортировка по убыванию математического ожидания $V11$, после чего выбираются N лучших интервалов. Финальная сортировка осуществляется по убыванию общей суммы `Amount`.

Результаты записываются в CSV-файл, содержащий границы интервалов, вычисленные статистики и количество записей. Завершающим этапом является освобождение ресурсов MPI и фиксация общего времени выполнения приложения.

6.8 Проблемы, выявленные при выполнении работы

В процессе разработки программного средства был выявлен ряд архитектурных и алгоритмических проблем, связанных как с выбором модели параллельных вычислений, так и с особенностями эксплуатации гетерогенного виртуального кластера.

Одной из ключевых проблем на начальном этапе являлось отсутствие механизма динамического распределения вычислительных ресурсов. Первоначальная версия программы использовала статическое, жёстко заданное распределение входного набора данных между вычислительными узлами кластера.

6.8.1 Проблема статического распределения ресурсов

Изначально разбиение входного набора данных выполнялось по фиксированному правилу, при котором заранее определённые доли данных назначались конкретным узлам:

- `ayzekcpu1` — 10 частей данных;
- `ayzekcpu2` — 11 частей данных;
- `ayzekgpu1` — 13 частей данных;
- `ayzekgpu2` — 14 частей данных.

Данное распределение было захардкожено в программном коде и не учитывало фактическое состояние вычислительных ресурсов во время выполнения. В частности, при отключении одного из GPU-ускорителей, уменьшении числа доступных ядер CPU или исключении одного из узлов из расчёта происходило нарушение логики распределения данных. Это приводило к некорректным результатам вычислений либо к аварийному завершению программы.

Таким образом, исходная архитектура обладала следующими недостатками:

- жёсткая привязка к конкретной конфигурации кластера;
- невозможность масштабирования;
- отсутствие отказоустойчивости при изменении доступных ресурсов;
- высокая вероятность логических ошибок при частичной деградации системы.

6.8.2 Проблема выбора корректной модели параллелизации

Отдельной сложностью являлся выбор эффективной модели параллелизации вычислений в условиях гетерогенного кластера. На начальном этапе были рассмотрены и реализованы несколько альтернативных подходов, которые впоследствии были признаны неэффективными.

6.8.3 Подход 2: Статическое распределение данных без учёта производительности GPU

Идея: равномерно распределить данные между всеми узлами кластера, игнорируя различия в производительности между CPU- и GPU-узлами. Каждый узел получает одинаковый объём данных для обработки независимо от своих аппаратных возможностей.

Недостатки:

- **Неэффективное использование GPU:** GPU-узлы, обладающие значительно большей вычислительной мощностью, завершают обработку своей части данных значительно быстрее и простаивают.
- **Дисбаланс времени выполнения:** Общее время выполнения приложения определяется самым медленным CPU-узлом.
- **Отсутствие адаптивности:** Невозможность перераспределения нагрузки при изменении конфигурации ресурсов.
- **Игнорирование аппаратной неоднородности:** Различия между моделями GPU и CPU полностью не учитываются.

6.8.4 Подход 3: Блокирующая отправка результатов с ожиданием главного узла

Идея: использовать блокирующие операции MPI (MPI_Send) для передачи результатов с рабочих узлов на главный процесс. Главный узел ожидает данные строго по порядку рангов.

Недостатки:

- **Последовательная обработка данных:** Главный узел не может начать обработку данных от следующего процесса до полного завершения обработки предыдущего.
- **Блокировка рабочих узлов:** Процессы простаивают в ожидании подтверждения получения данных.
- **Отсутствие перекрытия вычислений и коммуникаций:** Передача данных и обработка выполняются последовательно.
- **Зависимость от самого медленного процесса:** Один задержавшийся процесс блокирует обработку всех остальных результатов.

6.8.5 Подход 4: Последовательная обработка с барьерами синхронизации

Идея: использование барьеров синхронизации MPI между всеми этапами обработки данных.

Недостатки:

- **Низкая эффективность:** Быстрые узлы простаивают в ожидании медленных.
- **Высокие накладные расходы:** Частые барьеры существенно увеличивают общее время выполнения.
- **Отсутствие конвейеризации:** Невозможность перекрытия этапов обработки.
- **Сложность отладки и масштабирования.**

6.9 Финальное решение: гибридная параллельная архитектура с динамическим распределением

После обсуждения с научным руководителем было принято решение реализовать гибридную архитектуру, основанную на динамическом распределении нагрузки между MPI-процессами с учётом реальной производительности вычислительных узлов.

Ключевые особенности финального решения:

- поддержка динамического включения и отключения GPU-ускорителей;
- возможность управления числом используемых CPU-ядер (по 2 ядра на узел);
- масштабируемость от 1 до 8 вычислительных ядер;
- автоматическая адаптация к конфигурации кластера.

6.9.1 Идея параллелизации в финальной реализации

В финальной версии программы каждый MPI-процесс самостоятельно определяет наличие доступного GPU. На основе этой информации формируются весовые коэффициенты, характеризующие относительную производительность узла.

Изначально были выбраны следующие значения весов:

- GPU-узел — вес 1.28;
- CPU-узел — вес 1.0.

Весовые коэффициенты используются для пропорционального разбиения входного файла на байтовые диапазоны. Таким образом, GPU-узлы получают больший объём данных, а CPU-узлы — меньший, что позволяет выровнять общее время выполнения.

Каждый процесс выполняет полный цикл обработки своей части данных: чтение, фильтрацию, агрегацию и передачу локальных результатов. Обмен результатами осуществляется асинхронно, что позволяет перекрывать вычисления и коммуникации.

6.10 Особенности реализации

Одной из существенных особенностей реализации является использование компилятора nvcc версии 11.5. Данная версия содержит ограничения, не позволяющие корректно использовать библиотеку CUB, а также накладывает ограничения на использование стандартных средств CUDA Thrust.

Обновление драйверов NVIDIA и инструментов CUDA на виртуальных машинах под управлением Ubuntu Server оказалось трудоёмким и потенциально нестабильным процессом. В

связи с этим было принято решение реализовать GPU-часть приложения в виде отдельной динамической библиотеки с интерфейсом на языке C (`extern "C"`).

6.10.1 Особенности GPU-агрегации и сортировки

При включённом GPU агрегация и сортировка данных выполняются на графическом процессоре. Для сортировки использовался алгоритм пузырьковой сортировки, обладающий вычислительной сложностью $O(n^2)$.

Экспериментально было установлено, что время передачи и обработки результатов на финальном этапе (отправка данных на главный узел `ayzekcpu1`) для GPU-узлов достигает 3.56 секунды, в то время как для CPU-узлов это время составляет от 0.000 до 0.03 секунды.

Были предприняты попытки реализовать сортировку:

- с использованием radix sort на базе CUB;
- с использованием библиотеки Thrust.

Однако обе реализации оказались несовместимыми с используемой версией nvcc 11.5. После согласования с преподавателем было принято решение сохранить текущую реализацию и компенсировать её недостатки за счёт корректного подбора весов распределения нагрузки.

6.10.2 Балансировка нагрузки и отказ от OpenMP

Анализ показал, что при малом объёме данных GPU-узлы демонстрируют значительные накладные расходы, в то время как CPU-узлы обрабатывают такие объёмы данных быстрее. При увеличении объёма данных производительность CPU сначала возрастает линейно, а затем деградирует экспоненциально.

В связи с этим была проведена настройка весовых коэффициентов для GPU- и CPU-узлов, обеспечивающая более равномерную загрузку и минимизацию времени простоя ресурсов.

Использование OpenMP для дополнительной многопоточности на CPU рассматривалось, однако было решено отказаться от него во избежание значительного усложнения кода и переписывания уже реализованной архитектуры.

6.10.3 Правка весов

После того, как было замечено, что главный CPU узел относительно долго ждет данные от GPU, было принято решение изменить веса узла с GPU и без него. В окончательной реализации были установлены параметры CPU: 1.0, GPU: 0.2. Таким образом CPU принимает больше данных чем GPU, и наоборот GPU принимает меньше данных чем на CPU. Так как процесс сортировки и агрегации данных занимает больше времени на узле с GPU.

Итоговое соотношение:

- Один узел CPU: 100%
- Два узла CPU: 50% / 50%
- Три узла (один гетерогенный): 45.45%, 45.45%, 9.09%
- Четыре узла (два гетерогенных): 41.67%, 41.67%, 8.33%, 8.33%

7 Результаты

В ходе выполнения курсовой работы были получены результаты, представленные в таблице 7. Все измерения проводились на датасете, содержащем более 568 000 записей финансовых транзакций. Для каждой конфигурации использовались оптимальные параметры взвешенного распределения нагрузки: `CPU_WEIGHT` = 1.0 и `GPU_WEIGHT` = 0.2, что обеспечивало минимальный простой вычислительных узлов.

Конфигурация	Описание	Потоки	Время, с
1 CPU	Один CPU-узел	1	0.605
1 CPU	Один CPU-узел	2	0.627
2 CPU	Два CPU-узла	1	0.344
2 CPU	Два CPU-узла	2	0.339
3 CPU (+1 GPU)	Два CPU-узла, один GPU-узел	1	0.486
3 CPU (+1 GPU)	Два CPU-узла, один GPU-узел	2	0.485
4 CPU (+2 GPU)	Два CPU-узла, два GPU-узла	1	0.593
4 CPU (+2 GPU)	Два CPU-узла, два GPU-узла	2	0.574
3 CPU (-GPU)	Три CPU-узла (GPU отключён)	1	0.239
3 CPU (-GPU)	Три CPU-узла (GPU отключён)	2	0.242
4 CPU (-GPU)	Четыре CPU-узла (GPU отключён)	1	0.220
4 CPU (-GPU)	Четыре CPU-узла (GPU отключён)	2	0.197

Таблица 2: Результаты измерения времени выполнения для различных конфигураций кластера

Переход от конфигурации с одним CPU-узлом (1 поток) к конфигурации с двумя CPU-узлами (1 поток) обеспечивает ускорение в 1.76 раз ($0.605 / 0.344$). Использование двух потоков на двух CPU-узлах даёт незначительное улучшение производительности — ускорение в 1.01 раз по сравнению с однопоточной конфигурацией ($0.344 / 0.339$).

Конфигурация с тремя узлами, включающая один GPU-узел, демонстрирует снижение производительности по сравнению с двумя CPU-узлами: время выполнения увеличивается в 1.41 раз ($0.486 / 0.344$) для однопоточной версии. Это объясняется тем, что процесс сортировки и агрегации данных на GPU-узле занимает значительно больше времени из-за необходимости выполнения сортировки на центральном процессоре перед передачей данных на графический ускоритель. В частности, этап сортировки и группировки данных на CPU занимает около 240–290 мс, что существенно превышает время выполнения аналогичных операций на CPU-узлах (3–4 мс).

Добавление второго GPU-узла в конфигурацию с четырьмя узлами приводит к дальнейшему увеличению времени выполнения до 0.593 с (однопоточная версия) и 0.574 с (многопоточная версия), что в 1.72 раза медленнее конфигурации с двумя CPU-узлами. Это подтверждает, что при текущей реализации GPU-ускорение не обеспечивает преимущества для данной задачи из-за ограничений, связанных с версией CUDA Toolkit.

Конфигурации без использования GPU демонстрируют наилучшую производительность. Переход от двух CPU-узлов к трём CPU-узлам (GPU отключён, 1 поток) обеспечивает ускорение в 1.44 раз ($0.344 / 0.239$). Конфигурация с четырьмя CPU-узлами (GPU отключён, 2 потока) показывает наилучший результат — время выполнения 0.197 с, что в 3.07 раз быстрее базовой конфигурации с одним CPU-узлом ($0.605 / 0.197$).

Важным аспектом реализации является ограничение, связанное с версией CUDA Toolkit 11.5, установленной в виртуальном кластере. Библиотека CUB (CUDA Unbound), предоставляющая оптимизированные примитивы для сортировки и редукции на GPU, недоступна для данной версии компилятора NVCC. Обновление драйверов графических ускорителей не представлялось возможным в рамках текущей инфраструктуры. В связи с этим для всех конфигураций с GPU была использована реализация, выполняющая сортировку данных на центральном процессоре

перед передачей на графический ускоритель, что существенно снижает эффективность использования GPU-ресурсов.

Для каждой конфигурации с гетерогенными узлами были подобраны оптимальные параметры взвешенного распределения нагрузки: `CPU_WEIGHT = 1.0` и `GPU_WEIGHT = 0.2`. Данное соотношение весов обеспечивает минимальный простой вычислительных узлов, компенсируя более длительное время обработки данных на GPU-узлах за счёт уменьшения объёма данных, распределяемых на графические ускорители. В результате CPU-узлы получают приблизительно в 5 раз больше данных для обработки, чем GPU-узлы, что позволяет достичь баланса между загрузкой различных типов вычислительных ресурсов.

8 Заключение

В рамках курсовой работы была разработана и реализована виртуальная вычислительная система на базе гипервизора Hyper-V, объединяющая четыре виртуальных узла (два CPU-узла и два гетерогенных узла с GPU) под управлением Ubuntu Server 22.04. На кластере была развёрнута система управления рабочей нагрузкой SLURM, настроена сетевая файловая система NFS для совместного доступа к данным и реализована поддержка CUDA для GPU-ускорения вычислений.

Разработано параллельное приложение для анализа финансовых транзакций, обрабатывающее датасет объёмом более 568 000 записей. Приложение выполняет фильтрацию записей по условию $\text{Amount} \geq 9000$, группировку данных по интервалам признака V1 с шагом 0.1 и формирует итоговый отчёт с 50 группами, имеющими наибольшее математическое ожидание V11 среди групп с максимальным количеством отрицательных значений V7.

Реализована гибридная архитектура параллельных вычислений, сочетающая распределённую обработку данных между узлами кластера с использованием стандарта MPI и внутриузловое ускорение на графических процессорах с применением технологии CUDA. Ключевой особенностью решения является динамическое взвешенное распределение вычислительной нагрузки между узлами с учётом их аппаратных характеристик, что обеспечивает минимальный простой вычислительных ресурсов.

В ходе экспериментальных исследований были получены следующие результаты. Базовая конфигурация с одним CPU-узлом и одним потоком показала время выполнения 0.605 с. Оптимальная конфигурация с четырьмя CPU-узлами и восемью потоками (по два потока на каждый узел) продемонстрировала наилучшую производительность — время выполнения 0.197 с, что обеспечивает ускорение в 3.07 раз по сравнению с базовой конфигурацией.

Конфигурация с четырьмя узлами, включающая два CPU-узла и два GPU-узла, показала время выполнения 0.593 с при использовании одного потока на узел и 0.574 с при использовании двух потоков на узел. Данные результаты свидетельствуют о том, что при текущей реализации GPU-ускорение не обеспечивает преимущества для данной задачи.

Важным ограничением реализации явилась невозможность использования оптимизированных библиотек для сортировки данных на GPU. Были предприняты попытки реализации сортировки с использованием radix sort на базе библиотеки CUB (CUDA Unbound) и библиотеки Thrust, однако обе реализации оказались несовместимыми с версией компилятора NVCC 11.5, установленной в виртуальном кластере. Обновление драйверов графических ускорителей и инструментов CUDA не представлялось возможным в рамках текущей инфраструктуры. В связи с этим была использована реализация, выполняющая сортировку данных на центральном процессоре перед передачей на графический ускоритель, что существенно снижает эффективность использования GPU-ресурсов и приводит к увеличению времени обработки данных на GPU-узлах до 240–290 мс по сравнению с 3–4 мс на CPU-узлах.

Для компенсации указанных ограничений были подобраны оптимальные параметры взвешенного распределения нагрузки: $\text{CPU_WEIGHT} = 1.0$ и $\text{GPU_WEIGHT} = 0.2$. Данное соотношение обеспечивает минимальный простой вычислительных узлов, распределяя на CPU-узлы приблизительно в 5 раз больше данных, чем на GPU-узлы.

Итогом работы является успешная демонстрация возможности создания высокопроизводительных вычислительных систем на базе виртуальных машин с использованием доступного оборудования. Разработанное программное средство обеспечивает эффективную обработку больших объёмов данных в гетерогенных кластерах, демонстрируя ускорение более чем в 3 раза при использовании оптимальной конфигурации. Полученный опыт разработки и настройки виртуального кластера, а также созданная программно-аппаратная среда могут быть эффективно использованы в учебном процессе для изучения современных технологий параллельного и распределённого программирования.

Список литературы

- [1] ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. Структура и правила оформления.
- [2] MPI: A Message-Passing Interface Standard. Version 3.1. — [Электронный ресурс]. — MPI Forum, 2015. — URL: <https://www.mpi-forum.org/> (дата обращения: 24.12.2025).
- [3] NVIDIA CUDA C Programming Guide. Version 12. — [Электронный ресурс]. — NVIDIA Corporation, 2023. — URL: <https://docs.nvidia.com/cuda/> (дата обращения: 10.01.2026).
- [4] Hyper-V Virtualization Infrastructure. — [Электронный ресурс]. — Microsoft Docs, 2025. — URL: <https://learn.microsoft.com/en-us/virtualization/hyper-v-on-windows/> (дата обращения: 11.01.2026).
- [5] Linux Kernel Documentation. — [Электронный ресурс]. — The Linux Foundation, 2025. — URL: <https://www.kernel.org/doc/html/latest/> (дата обращения: 10.01.2026).
- [6] Credit Card Fraud Detection Dataset (2023). — [Электронный ресурс]. — Kaggle Datasets. — URL: <https://www.kaggle.com/datasets/nelgiryewithana/credit-card-fraud-detection-dataset-2023> (дата обращения: 24.12.2025).
- [7] Slurm Workload Manager Documentation. — [Электронный ресурс]. — SchedMD, 2025. — URL: <https://slurm.schedmd.com/documentation.html> (дата обращения: 15.01.2026).

Приложение А

Исходный код файла SLURM.

Листинг 1: Файл run.slurm

```
1  #!/bin/bash
2  #SBATCH --job-name=creditcard
3  #SBATCH --nodes=4
4  #SBATCH --ntasks=4
5  #SBATCH --cpus-per-task=2
6  #SBATCH --nodelist=ayzekcpu1,ayzekcpu2,ayzekgpu1,ayzekgpu2
7  #SBATCH --output=out.txt
8  #SBATCH --export=ALL
9
10 export DATA\_PATH="/home/ayzek/creditcard\_2023.csv"
11 export GPU\_PROCESS\_WEIGHT=${GPU\_PROCESS\_WEIGHT:-0.2}
12 export CPU\_PROCESS\_WEIGHT=${CPU\_PROCESS\_WEIGHT:-1.0}
13 export USE\_CUDA=1
14 export USE\_BLOCK\_KERNEL=0
15
16 module load cuda 2>/dev/null || true
17
18 if [ -d "$HOME/cuda/bin" ]; then
19     export PATH=$HOME/cuda/bin:$PATH
20     export LD\_LIBRARY\_PATH=$HOME/cuda/lib64:$LD\_LIBRARY\_PATH
21 fi
22
23 cd /mnt/share/supercomputers
24
25 TOP\_N=${TOP\_N:-50}
26 echo "Using TOP\_N = $TOP\_N"
27
28 APP\_PATH="/mnt/share/supercomputers/build/bitcoin\_app"
29
30 mpirun -np $SLURM\_NTASKS -x USE\_CUDA -x DATA\_PATH -x READ\_OVERLAP\_BYTES
    -x AGGREGATION\_INTERVAL -x USE\_BLOCK\_KERNEL -x GPU\_PROCESS\_WEIGHT -
    x CPU\_PROCESS\_WEIGHT $APP\_PATH -n $TOP\_N
```

Приложение В

Исходный код файла сборки проекта.

Листинг 2: Файл Makefile

```
1      CXX = mpic++
2  CXXFLAGS = -std=c++17 -O2 -Wall -Wextra -Wno-cast-function-type -fopenmp
3
4  NVCC = nvcc
5  NVCCFLAGS = -O3 -arch=sm\_86 -Xcompiler -fPIC --expt-relaxed-constexpr -
      Xcompiler -Wno-deprecated-declarations
6
7  SRC\_DIR = src
8  BUILD\_DIR = build
9
10 SRC = $(wildcard $(SRC\_DIR)/*.cpp)
11 OBJ = $(patsubst $(SRC\_DIR)/%.cpp,$(BUILD\_DIR)/%.o,$(SRC))
12
13 TARGET = $(BUILD\_DIR)/bitcoin\_app
14
15 PLUGIN\_SRC = $(SRC\_DIR)/gpu\_plugin.cu
16 PLUGIN = $(BUILD\_DIR)/libgpu\_compute.so
17
18 all: $(TARGET) $(PLUGIN)
19
20 $(BUILD\_DIR):
21     mkdir -p $(BUILD\_DIR)
22
23 $(BUILD\_DIR)/%.o: $(SRC\_DIR)/%.cpp | $(BUILD\_DIR)
24     $(CXX) $(CXXFLAGS) -c $< -o $@
25
26 $(TARGET): $(OBJ) | $(BUILD\_DIR)
27     $(CXX) $(CXXFLAGS) $^ -o $@ -ldl
28
29 $(PLUGIN): $(PLUGIN\_SRC) | $(BUILD\_DIR)
30     @echo "Building GPU plugin on main node..."
31     @echo "nvcc 11.5 has a known bug with C++ stdlib. Trying workarounds
    ..."
32     @if $(NVCC) -O3 -arch=sm\_86 -shared \
33         -Xcompiler -fPIC \
34         -Xcompiler -std=c++11 \
35         --compiler-options -fno-strict-aliasing \
36         --compiler-options -Wno-deprecated-declarations \
37         --expt-relaxed-constexpr \
38         -D\_GNUC\_ =7 \
39         -D\_GNUC\_MINOR\_ =5 \
40         $< -o $(PLUGIN) 2>&1; then \
41         echo "GPU plugin built successfully with workaround flags";
42     \
43     fi
44
45 gpu-plugin: $(PLUGIN)
46
47 clean:
48     rm -rf $(BUILD\_DIR)
49
50 run: all
51     sbatch run.slurm
52
```

53 | .PHONY: all clean run gpu-plugin

Приложение С

Исходный код файла агрегации.

Листинг 3: aggregation.cpp

```
1  #include "aggregation.hpp"
2  #include "utils.hpp"
3
4  #include <algorithm>
5  #include <cstdint>
6  #include <limits>
7  #include <vector>
8  #include <cmath>
9  #include <map>
10
11 std::vector<PeriodStats> aggregate\_periods(const std::vector<Record>&
    records) {
12     const double interval = 0.1;
13
14     std::vector<PeriodStats> result;
15     if (records.empty()) return result;
16
17     struct PeriodAccumulator {
18         double v1\_min = std::numeric\_limits<double>::max();
19         double v1\_max = std::numeric\_limits<double>::lowest();
20         double v11\_sum = 0.0;
21         int64\_t negative\_v7\_count = 0;
22         int64\_t zero\_v7\_count = 0;
23         double amount\_sum = 0.0;
24         int64\_t count = 0;
25
26         void add(const Record& r) {
27             v1\_min = std::min(v1\_min, r.v1);
28             v1\_max = std::max(v1\_max, r.v1);
29             v11\_sum += r.v11;
30             if (r.v7 < 0.0) {
31                 negative\_v7\_count++;
32             } else if (r.v7 == 0.0) {
33                 zero\_v7\_count++;
34             }
35             amount\_sum += r.amount;
36             ++count;
37         }
38     };
39
40
41     std::map<PeriodIndex, PeriodAccumulator> groups;
42
43     for (const auto& r : records) {
44
45         PeriodIndex period = static\_cast<PeriodIndex>(std::floor(r.v1 /
    interval));
46         groups[period].add(r);
47     }
48
49
50     bool has\_negative = false;
51     for (const auto& [period, acc] : groups) {
52         if (acc.negative\_v7\_count > 0) {
53             has\_negative = true;
```



```

54         break;
55     }
56 }
57
58 for (const auto& [period, acc] : groups) {
59     PeriodStats stats;
60     stats.period = period;
61     stats.v1\_min = acc.v1\_min;
62     stats.v1\_max = acc.v1\_max;
63     stats.v11\_sum = acc.v11\_sum;
64     if (!has\_negative && acc.negative\_v7\_count == 0) {
65         stats.negative\_v7\_count = acc.zero\_v7\_count;
66     } else {
67         stats.negative\_v7\_count = acc.negative\_v7\_count;
68     }
69     stats.amount\_sum = acc.amount\_sum;
70     stats.count = acc.count;
71     result.push\_back(stats);
72 }
73
74 std::sort(result.begin(), result.end(),
75     [](const PeriodStats& a, const PeriodStats& b) {
76         return a.period < b.period;
77     });
78
79 return result;
80 }

```

Приложение D

Исходный код файла разбиения интервалов.

Листинг 4: intervals.cpp

```
1  #include "aggregation.hpp"
2  #include "utils.hpp"
3
4  #include <algorithm>
5  #include <cstdint>
6  #include <limits>
7  #include <vector>
8  #include <cmath>
9  #include <map>
10
11 std::vector<PeriodStats> aggregate\_periods(const std::vector<Record>&
12     records) {
13     const double interval = 0.1;
14
15     std::vector<PeriodStats> result;
16     if (records.empty()) return result;
17
18     struct PeriodAccumulator {
19         double v1\_min = std::numeric\_limits<double>::max();
20         double v1\_max = std::numeric\_limits<double>::lowest();
21         double v11\_sum = 0.0;
22         int64\_t negative\_v7\_count = 0;
23         int64\_t zero\_v7\_count = 0;
24         double amount\_sum = 0.0;
25         int64\_t count = 0;
26
27         void add(const Record& r) {
28             v1\_min = std::min(v1\_min, r.v1);
29             v1\_max = std::max(v1\_max, r.v1);
30             v11\_sum += r.v11;
31             if (r.v7 < 0.0) {
32                 negative\_v7\_count++;
33             } else if (r.v7 == 0.0) {
34                 zero\_v7\_count++;
35             }
36             amount\_sum += r.amount;
37             ++count;
38         }
39     };
40
41     std::map<PeriodIndex, PeriodAccumulator> groups;
42
43     for (const auto& r : records) {
44         PeriodIndex period = static\_cast<PeriodIndex>(std::floor(r.v1 /
45     interval));
46         groups[period].add(r);
47     }
48
49     bool has\_negative = false;
50     for (const auto& [period, acc] : groups) {
51         if (acc.negative\_v7\_count > 0) {
52             has\_negative = true;
53         }
```

```

54         break;
55     }
56 }
57
58
59 for (const auto& [period, acc] : groups) {
60     PeriodStats stats;
61     stats.period = period;
62     stats.v1\_min = acc.v1\_min;
63     stats.v1\_max = acc.v1\_max;
64     stats.v11\_sum = acc.v11\_sum;
65
66     if (!has\_negative && acc.negative\_v7\_count == 0) {
67         stats.negative\_v7\_count = acc.zero\_v7\_count;
68     } else {
69         stats.negative\_v7\_count = acc.negative\_v7\_count;
70     }
71     stats.amount\_sum = acc.amount\_sum;
72     stats.count = acc.count;
73     result.push\_back(stats);
74 }
75
76
77 std::sort(result.begin(), result.end(),
78     [](const PeriodStats& a, const PeriodStats& b) {
79         return a.period < b.period;
80     });
81
82 return result;
83 }

```

Приложение Е

Исходный код файла чтений и работой с датасетом.

Листинг 5: csv_loader.cpp

```
1      #include "csv\_loader.hpp"
2      #include <fstream>
3      #include <sstream>
4      #include <iostream>
5      #include <stdexcept>
6      #include <numeric>
7
8      bool parse\_csv\_line(const std::string& line, Record& record) {
9          if (line.empty()) {
10             return false;
11         }
12
13         std::stringstream ss(line);
14         std::string item;
15
16         try {
17
18             if (!std::getline(ss, item, ',')) return false;
19
20
21             if (!std::getline(ss, item, ',')) return false;
22             record.v1 = std::stod(item);
23
24
25             for (int i = 0; i < 5; i++) {
26                 if (!std::getline(ss, item, ',')) return false;
27             }
28
29
30             if (!std::getline(ss, item, ',')) return false;
31             record.v7 = std::stod(item);
32
33
34             for (int i = 0; i < 3; i++) {
35                 if (!std::getline(ss, item, ',')) return false;
36             }
37
38
39             if (!std::getline(ss, item, ',')) return false;
40             record.v11 = std::stod(item);
41
42
43             for (int i = 0; i < 17; i++) {
44                 if (!std::getline(ss, item, ',')) return false;
45             }
46
47
48             if (!std::getline(ss, item, ',')) return false;
49             record.amount = std::stod(item);
50
51
52             if (record.amount < 9000.00) {
53                 return false;
54             }
55         }
```

```

56         return true;
57     } catch (const std::exception&) {
58         return false;
59     }
60 }
61
62 std::vector<Record> load\_csv\_parallel(int rank, int size, const std::
vector<int>\& provided\_shares) {
63     std::vector<Record> data;
64
65
66     std::string data\_path = get\_data\_path();
67     std::vector<int> shares;
68
69
70     if (!provided\_shares.empty() &\& provided\_shares.size() == static\
\_cast<size\_t>(size)) {
71         shares = provided\_shares;
72         if (rank == 0) {
73             std::cout << "Rank 0: Using provided process-based shares (
weighted balancing)" << std::endl;
74         }
75     } else {
76         shares = get\_data\_read\_shares();
77         if (rank == 0) {
78             std::cout << "Rank 0: Using auto-calculated node-based shares (
fallback)" << std::endl;
79         }
80     }
81
82     int64\_t overlap\_bytes = get\_read\_overlap\_bytes();
83
84
85     if (rank == 0) {
86         std::cout << "Rank 0: Data distribution shares: ";
87         for (size\_t i = 0; i < shares.size(); i++) {
88             std::cout << shares[i];
89             if (i < shares.size() - 1) std::cout << ",";
90         }
91         std::cout << " (total: " << std::accumulate(shares.begin(), shares.
end(), 0) << ")" << std::endl;
92     }
93
94
95     int64\_t file\_size = get\_file\_size(data\_path);
96
97
98     ByteRange range = calculate\_byte\_range(rank, size, file\_size, shares,
overlap\_bytes);
99
100
101     std::ifstream file(data\_path, std::ios::binary);
102     if (!file.is\_open()) {
103         throw std::runtime\_error("Cannot open file: " + data\_path);
104     }
105
106
107     file.seekg(range.start);
108
109
110     int64\_t bytes\_to\_read = range.end - range.start;

```

```

111     std::vector<char> buffer(bytes\_to\_read);
112     file.read(buffer.data(), bytes\_to\_read);
113     int64\_t bytes\_read = file.gcount();
114
115     file.close();
116
117
118     std::string content(buffer.data(), bytes\_read);
119
120
121     size\_t parse\_start = 0;
122     if (rank == 0) {
123
124         size\_t header\_end = content.find('\n');
125         if (header\_end != std::string::npos) {
126             parse\_start = header\_end + 1;
127         }
128     } else {
129
130         size\_t first\_newline = content.find('\n');
131         if (first\_newline != std::string::npos) {
132             parse\_start = first\_newline + 1;
133         }
134     }
135
136
137     size\_t parse\_end = content.size();
138     if (rank != size - 1) {
139
140         size\_t last\_newline = content.rfind('\n');
141         if (last\_newline != std::string::npos && last\_newline > parse\_
142 _start) {
143             parse\_end = last\_newline;
144         }
145     }
146
147     size\_t pos = parse\_start;
148     while (pos < parse\_end) {
149         size\_t line\_end = content.find('\n', pos);
150         if (line\_end == std::string::npos || line\_end > parse\_end) {
151             line\_end = parse\_end;
152         }
153
154         std::string line = content.substr(pos, line\_end - pos);
155
156
157         if (!line.empty() && line.back() == '\r') {
158             line.pop\_back();
159         }
160
161         Record record;
162         if (parse\_csv\_line(line, record)) {
163             data.push\_back(record);
164         }
165
166
167         pos = line\_end + 1;
168     }
169
170     return data;

```


Приложение F

Исходный код файла работы с GPU.

Листинг 6: gpu_loader.cpp

```
1      #include "gpu\_loader.hpp"
2      #include "period\_stats.hpp"
3      #include "record.hpp"
4      #include <dlfcn.h>
5      #include <iostream>
6      #include <cstdlib>
7      #include <cmath>
8      #include <cstdliblib>
9      #include <string>
10     #include <vector>
11     #include <algorithm>
12     #include <map>
13
14
15     struct GpuPeriodStats {
16         int64\_t period;
17         double v1\_min;
18         double v1\_max;
19         double v11\_sum;
20         int64\_t negative\_v7\_count;
21         double amount\_sum;
22         int64\_t count;
23     };
24
25
26     using gpu\_is\_available\_fn = int (*)( );
27
28     using gpu\_aggregate\_periods\_fn = int (*)(
29         const double* h\_v1,
30         const double* h\_v7,
31         const double* h\_v11,
32         const double* h\_amount,
33         int num\_records,
34         double interval,
35         GpuPeriodStats** h\_out\_stats,
36         int* out\_num\_periods
37     );
38
39     using gpu\_free\_results\_fn = void (*)(GpuPeriodStats*);
40
41     static void* gpu\_lib\_handle = nullptr;
42
43     static void* get\_gpu\_lib\_handle() {
44         if (gpu\_lib\_handle == nullptr) {
45
46             const char* home = getenv("HOME");
47             std::string lib\_paths[] = {
48                 "./libgpu\_compute.so",
49                 "~/libgpu\_compute.so",
50                 home ? std::string(home) + "/libgpu\_compute.so" : "",
51                 "/mnt/share/supercomputers/build/libgpu\_compute.so"
52             };
53
54             for (const auto& path : lib\_paths) {
55                 if (path.empty()) continue;
```



```

56         std::string expanded\_path = path;
57         if (expanded\_path[0] == '~' && home) {
58             expanded\_path = std::string(home) + expanded\_path.substr
(1);
59         }
60         gpu\_lib\_handle = dlopen(expanded\_path.c\_str(), RTLD\_LAZY);
61         if (gpu\_lib\_handle) {
62             return gpu\_lib\_handle;
63         }
64     }
65
66     return nullptr;
67 }
68 return gpu\_lib\_handle;
69 }
70
71 bool gpu\_is\_available() {
72     void* handle = get\_gpu\_lib\_handle();
73     if (!handle) {
74         return false;
75     }
76
77
78     gpu\_is\_available\_fn fn = (gpu\_is\_available\_fn)dlsym(handle, "gpu\_
\_is\_available");
79     if (!fn) {
80         return false;
81     }
82
83     int result = fn();
84     return result != 0;
85 }
86
87 bool aggregate\_periods\_gpu(
88     const std::vector<Record>& records,
89     int64\_t aggregation\_interval,
90     std::vector<PeriodStats>& out\_stats)
91 {
92     if (records.empty()) {
93         out\_stats.clear();
94         return true;
95     }
96
97     void* handle = get\_gpu\_lib\_handle();
98     if (!handle) {
99         return false;
100     }
101
102
103     gpu\_aggregate\_periods\_fn aggregate\_fn = (gpu\_aggregate\_periods\_fn)
dlsym(handle, "gpu\_aggregate\_periods");
104     gpu\_free\_results\_fn free\_fn = (gpu\_free\_results\_fn)dlsym(handle,
"gpu\_free\_results");
105
106     if (!aggregate\_fn || !free\_fn) {
107         return false;
108     }
109
110
111     std::vector<double> h\_v1(records.size());
112     std::vector<double> h\_v7(records.size());

```

```

113     std::vector<double> h\_v1(records.size());
114     std::vector<double> h\_amount(records.size());
115
116     for (size\_t i = 0; i < records.size(); i++) {
117         h\_v1[i] = records[i].v1;
118         h\_v7[i] = records[i].v7;
119         h\_v11[i] = records[i].v11;
120         h\_amount[i] = records[i].amount;
121     }
122
123
124
125     const double v1\_interval = 0.1;
126     GpuPeriodStats* gpu\_stats = nullptr;
127     int num\_periods = 0;
128
129     int result = aggregate\_fn(
130         h\_v1.data(),
131         h\_v7.data(),
132         h\_v11.data(),
133         h\_amount.data(),
134         static\_cast<int>(records.size()),
135         v1\_interval,
136         &gpu\_stats,
137         &num\_periods
138     );
139
140     if (result != 0 || gpu\_stats == nullptr) {
141         return false;
142     }
143
144
145     out\_stats.clear();
146     out\_stats.reserve(num\_periods);
147
148
149     bool has\_negative = false;
150     for (size\_t i = 0; i < records.size(); i++) {
151         if (records[i].v7 < 0.0) {
152             has\_negative = true;
153             break;
154         }
155     }
156
157
158     std::map<PeriodIndex, int64\_t> zero\_v7\_by\_period;
159     if (!has\_negative) {
160         for (size\_t i = 0; i < records.size(); i++) {
161             if (records[i].v7 == 0.0) {
162                 PeriodIndex period = static\_cast<PeriodIndex>(std::floor(
records[i].v1 / 0.1));
163                 zero\_v7\_by\_period[period]++;
164             }
165         }
166     }
167
168     for (int i = 0; i < num\_periods; i++) {
169         PeriodStats ps;
170         ps.period = gpu\_stats[i].period;
171         ps.v1\_min = gpu\_stats[i].v1\_min;
172         ps.v1\_max = gpu\_stats[i].v1\_max;

```

```

173         ps.v11\_sum = gpu\_stats[i].v11\_sum;
174
175         if (!has\_negative &\& gpu\_stats[i].negative\_v7\_count == 0) {
176             auto it = zero\_v7\_by\_period.find(gpu\_stats[i].period);
177             ps.negative\_v7\_count = (it != zero\_v7\_by\_period.end()) ? it
->second : 0;
178         } else {
179             ps.negative\_v7\_count = gpu\_stats[i].negative\_v7\_count;
180         }
181         ps.amount\_sum = gpu\_stats[i].amount\_sum;
182         ps.count = gpu\_stats[i].count;
183         out\_stats.push\_back(ps);
184     }
185
186
187     free\_fn(gpu\_stats);
188
189     return true;
190 }

```

Приложение G

Исходный код файла плагина GPU на CUDA.

Листинг 7: gpu_plugin.cu

```
1  #include <cuda\_runtime.h>
2  #include <cstdint>
3  #include <cfloat>
4  #include <cstdio>
5  #include <cstdlib>
6  #include <ctime>
7  #include <cmath>
8  #include <cstring>
9  #include <climits>
10
11
12
13
14
15
16  struct GpuPeriodStats {
17      int64\_t period;
18      double v1\_min;
19      double v1\_max;
20      double v11\_sum;
21      int64\_t negative\_v7\_count;
22      double amount\_sum;
23      int64\_t count;
24  };
25
26
27
28
29
30  static double get\_time\_ms() {
31      struct timespec ts;
32      clock\__gettime(CLOCK\_MONOTONIC, &ts);
33      return ts.tv\_sec * 1000.0 + ts.tv\_nsec / 1000000.0;
34  }
35
36  #define CUDA\_CHECK(call) do { \
37      cudaError\_t err = call; \
38      if (err != cudaSuccess){ \
39          printf("CUDA error at %s:%d: %s\n", \_\_FILE\_\_, \_\_LINE\_\_,
40              cudaGetErrorString(err)); \
41          return -1; \
42      } \
43  } while(0)
44
45
46
47
48
49  \_\_global\_\_ void compute\_period\_ids\_kernel(
50      const double* \_\_restrict\_\_ v1,
51      int64\_t* \_\_restrict\_\_ period\_ids,
52      int n,
53      double interval)
54  {
```

```

55     int idx = blockIdx.x * blockDim.x + threadIdx.x;
56     if (idx < n) {
57         period\_ids[idx] = static\_cast<int64\_t>(floor(v1[idx] / interval))
58     }
59 }
60
61
62
63
64
65 \_\_global\_\_ void init\_indices\_kernel(int* indices, int n) {
66     int idx = blockIdx.x * blockDim.x + threadIdx.x;
67     if (idx < n) {
68         indices[idx] = idx;
69     }
70 }
71
72
73
74
75
76 \_\_global\_\_ void copy\_sorted\_period\_ids\_kernel(
77     const int64\_t* \_\_restrict\_\_ input\_period\_ids,
78     const int* \_\_restrict\_\_ sorted\_indices,
79     int64\_t* \_\_restrict\_\_ output\_period\_ids,
80     int n)
81 {
82     int idx = blockIdx.x * blockDim.x + threadIdx.x;
83     if (idx < n) {
84         output\_period\_ids[idx] = input\_period\_ids[sorted\_indices[idx]];
85     }
86 }
87
88
89
90
91
92
93 \_\_global\_\_ void radix\_count\_kernel(
94     const int64\_t* \_\_restrict\_\_ keys,
95     const int* \_\_restrict\_\_ values,
96     int* \_\_restrict\_\_ counts,
97     int n,
98     int byte\_shift)
99 {
100     int idx = blockIdx.x * blockDim.x + threadIdx.x;
101     if (idx >= n) return;
102
103
104     int byte\_val = (int)((keys[idx] >> (byte\_shift * 8)) & 0xFF);
105
106
107     \_\_shared\_\_ int s\_counts[256];
108     if (threadIdx.x < 256) {
109         s\_counts[threadIdx.x] = 0;
110     }
111     \_\_syncthreads();
112
113     atomicAdd(&s\_counts[byte\_val], 1);
114     \_\_syncthreads();

```

```

115
116
117     if (threadIdx.x == 0) {
118         for (int i = 0; i < 256; i++) {
119             atomicAdd(&counts[i], s_counts[i]);
120         }
121     }
122 }
123
124
125 __global__ void radix_reorder_kernel(
126     const int64_t* __restrict input_keys,
127     const int* __restrict input_values,
128     int64_t* __restrict output_keys,
129     int* __restrict output_values,
130     const int* __restrict prefix_sum,
131     int n,
132     int byte_shift)
133 {
134     int idx = blockIdx.x * blockDim.x + threadIdx.x;
135     if (idx >= n) return;
136
137     int byte_val = (int)((input_keys[idx] >> (byte_shift * 8)) & 0xFF);
138     int new_pos = prefix_sum[byte_val] - 1;
139
140     output_keys[new_pos] = input_keys[idx];
141     output_values[new_pos] = input_values[idx];
142
143     atomicSub((int*)&prefix_sum[byte_val], 1);
144 }
145
146
147
148
149
150
151
152 __global__ void find_period_boundaries_kernel(
153     const int64_t* __restrict sorted_period_ids,
154     int* __restrict boundaries,
155     int n)
156 {
157     int idx = blockIdx.x * blockDim.x + threadIdx.x;
158
159     if (idx == 0 && n > 0) {
160         boundaries[0] = 1;
161     }
162
163     if (idx < n - 1) {
164         boundaries[idx + 1] = (sorted_period_ids[idx] != sorted_period_ids[idx + 1]) ? 1 : 0;
165     }
166 }
167
168
169
170
171
172
173
174 __global__ void group_periods_simple_kernel(

```

```

175     const int64_t* \_restrict\_sorted\_period\_ids,
176     const int* \_restrict\_boundaries,
177     int64_t* \_restrict\_unique\_periods,
178     int* \_restrict\_offsets,
179     int* \_restrict\_num\_periods\_out,
180     int n)
181 {
182     int idx = blockIdx.x * blockDim.x + threadIdx.x;
183
184
185     if (idx == 0 && n > 0) {
186         int pos = atomicAdd(num\_periods\_out, 1);
187         if (pos < n) {
188             unique\_periods[pos] = sorted\_period\_ids[0];
189             offsets[pos] = 0;
190         }
191     }
192
193
194     if (idx < n - 1 && boundaries[idx + 1] == 1) {
195         int pos = atomicAdd(num\_periods\_out, 1);
196         if (pos < n) {
197             unique\_periods[pos] = sorted\_period\_ids[idx + 1];
198             offsets[pos] = idx + 1;
199         }
200     }
201 }
202
203
204
205
206
207 \_global\_ void compute\_period\_counts\_kernel(
208     const int* \_restrict\_offsets,
209     int* \_restrict\_counts,
210     int num\_periods,
211     int total\_records)
212 {
213     int idx = blockIdx.x * blockDim.x + threadIdx.x;
214     if (idx < num\_periods) {
215         int start = offsets[idx];
216         int end = (idx + 1 < num\_periods) ? offsets[idx + 1] : total\_
\_records;
217         counts[idx] = end - start;
218     }
219 }
220
221
222
223
224
225
226
227
228
229
230 \_global\_ void aggregate\_periods\_kernel(
231     const double* \_restrict\_v1,
232     const double* \_restrict\_v7,
233     const double* \_restrict\_v11,
234     const double* \_restrict\_amount,

```

```

235     const int64_t* \_\_restrict\_\_ unique\_periods,
236     const int* \_\_restrict\_\_ offsets,
237     const int* \_\_restrict\_\_ counts,
238     int num\_periods,
239     GpuPeriodStats* \_\_restrict\_\_ out\_stats)
240 {
241     int period\_idx = blockIdx.x;
242     if (period\_idx >= num\_periods) return;
243
244     int offset = offsets[period\_idx];
245     int count = counts[period\_idx];
246
247
248     \_\_shared\_\_ double s\_v1\_min;
249     \_\_shared\_\_ double s\_v1\_max;
250     \_\_shared\_\_ double s\_v11\_sum;
251     \_\_shared\_\_ int64_t s\_negative\_v7\_count;
252     \_\_shared\_\_ double s\_amount\_sum;
253
254
255     if (threadIdx.x == 0) {
256         s\_v1\_min = DBL\_MAX;
257         s\_v1\_max = -DBL\_MAX;
258         s\_v11\_sum = 0.0;
259         s\_negative\_v7\_count = 0;
260         s\_amount\_sum = 0.0;
261     }
262     \_\_syncthreads();
263
264
265     double local\_v1\_min = DBL\_MAX;
266     double local\_v1\_max = -DBL\_MAX;
267     double local\_v11\_sum = 0.0;
268     int64_t local\_negative\_v7\_count = 0;
269     double local\_amount\_sum = 0.0;
270
271
272     for (int i = threadIdx.x; i < count; i += blockDim.x) {
273         int record\_idx = offset + i;
274         double v1\_val = v1[record\_idx];
275         double v7\_val = v7[record\_idx];
276         double v11\_val = v11[record\_idx];
277         double amount\_val = amount[record\_idx];
278
279         local\_v1\_min = fmin(local\_v1\_min, v1\_val);
280         local\_v1\_max = fmax(local\_v1\_max, v1\_val);
281         local\_v11\_sum += v11\_val;
282         if (v7\_val < 0.0) {
283             local\_negative\_v7\_count++;
284         }
285         local\_amount\_sum += amount\_val;
286     }
287
288
289     atomicMin(reinterpret\_cast<unsigned long long*>(&s\_v1\_min),
290              \_\_double\_as\_longlong(local\_v1\_min));
291     atomicMax(reinterpret\_cast<unsigned long long*>(&s\_v1\_max),
292              \_\_double\_as\_longlong(local\_v1\_max));
293     atomicAdd(&s\_v11\_sum, local\_v11\_sum);
294     atomicAdd(reinterpret\_cast<unsigned long long*>(&s\_negative\_v7\_
_count),

```



```

295         static\_cast<unsigned long long>(local\_negative\_v7\_count));
296     atomicAdd(&s\_amount\_sum, local\_amount\_sum);
297
298     \_\_syncthreads();
299
300
301     if (threadIdx.x == 0) {
302         GpuPeriodStats stats;
303         stats.period = unique\_periods[period\_idx];
304         stats.v1\_min = s\_v1\_min;
305         stats.v1\_max = s\_v1\_max;
306         stats.v11\_sum = s\_v11\_sum;
307         stats.negative\_v7\_count = s\_negative\_v7\_count;
308         stats.amount\_sum = s\_amount\_sum;
309         stats.count = count;
310         out\_stats[period\_idx] = stats;
311     }
312 }
313
314
315
316
317
318
319 \_\_global\_\_ void aggregate\_periods\_simple\_kernel(
320     const double* \_\_restrict\_\_ v1,
321     const double* \_\_restrict\_\_ v7,
322     const double* \_\_restrict\_\_ v11,
323     const double* \_\_restrict\_\_ amount,
324     const int64\_t* \_\_restrict\_\_ unique\_periods,
325     const int* \_\_restrict\_\_ offsets,
326     const int* \_\_restrict\_\_ counts,
327     int num\_periods,
328     GpuPeriodStats* \_\_restrict\_\_ out\_stats)
329 {
330     int period\_idx = blockIdx.x * blockDim.x + threadIdx.x;
331     if (period\_idx >= num\_periods) return;
332
333     int offset = offsets[period\_idx];
334     int count = counts[period\_idx];
335
336     double v1\_min = DBL\_MAX;
337     double v1\_max = -DBL\_MAX;
338     double v11\_sum = 0.0;
339     int64\_t negative\_v7\_count = 0;
340     double amount\_sum = 0.0;
341
342     for (int i = 0; i < count; i++) {
343         int record\_idx = offset + i;
344         double v1\_val = v1[record\_idx];
345         double v7\_val = v7[record\_idx];
346         double v11\_val = v11[record\_idx];
347         double amount\_val = amount[record\_idx];
348
349         v1\_min = fmin(v1\_min, v1\_val);
350         v1\_max = fmax(v1\_max, v1\_val);
351         v11\_sum += v11\_val;
352         if (v7\_val < 0.0) {
353             negative\_v7\_count++;
354         }
355         amount\_sum += amount\_val;

```

```

356     }
357
358     GpuPeriodStats stats;
359     stats.period = unique\_periods[period\_idx];
360     stats.v1\_min = v1\_min;
361     stats.v1\_max = v1\_max;
362     stats.v11\_sum = v11\_sum;
363     stats.negative\_v7\_count = negative\_v7\_count;
364     stats.amount\_sum = amount\_sum;
365     stats.count = count;
366     out\_stats[period\_idx] = stats;
367 }
368
369
370
371
372
373 \_\_global\_\_ void reorder\_data\_kernel(
374     const double* \_\_restrict\_\_ src\_v1, const double* \_\_restrict\_\_
375     src\_v7,
376     const double* \_\_restrict\_\_ src\_v11, const double* \_\_restrict\_\_
377     src\_amount,
378     const int* \_\_restrict\_\_ indices,
379     double* \_\_restrict\_\_ dst\_v1, double* \_\_restrict\_\_ dst\_v7,
380     double* \_\_restrict\_\_ dst\_v11, double* \_\_restrict\_\_ dst\_amount,
381     int n)
382 {
383     int idx = blockIdx.x * blockDim.x + threadIdx.x;
384     if (idx < n) {
385         int src\_idx = indices[idx];
386         dst\_v1[idx] = src\_v1[src\_idx];
387         dst\_v7[idx] = src\_v7[src\_idx];
388         dst\_v11[idx] = src\_v11[src\_idx];
389         dst\_amount[idx] = src\_amount[src\_idx];
390     }
391 }
392
393
394
395 extern "C" int gpu\_is\_available() {
396     int n = 0;
397     cudaError\_t err = cudaGetDeviceCount(&n);
398     if (err != cudaSuccess) return 0;
399     return (n > 0) ? 1 : 0;
400 }
401
402
403
404
405
406
407 struct PeriodIndexPair {
408     int64\_t period;
409     int index;
410 };
411
412
413 static int compare\_period\_pairs(const void* a, const void* b) {
414     const PeriodIndexPair* pa = (const PeriodIndexPair*)a;

```

```

415     const PeriodIndexPair* pb = (const PeriodIndexPair*)b;
416     if (pa->period < pb->period) return -1;
417     if (pa->period > pb->period) return 1;
418     return 0;
419 }
420
421
422
423
424
425
426 extern "C" int gpu\_aggregate\_periods(
427     const double* h\_v1,
428     const double* h\_v7,
429     const double* h\_v11,
430     const double* h\_amount,
431     int num\_records,
432     double interval,
433     GpuPeriodStats** h\_out\_stats,
434     int* out\_num\_periods)
435 {
436     if (num\_records == 0) {
437         *h\_out\_stats = nullptr;
438         *out\_num\_periods = 0;
439         return 0;
440     }
441
442     double total\_start = get\_time\_ms();
443
444
445
446
447     double step1\_start = get\_time\_ms();
448
449     double* d\_v1 = nullptr;
450     double* d\_v7 = nullptr;
451     double* d\_v11 = nullptr;
452     double* d\_amount = nullptr;
453     int64\_t* d\_period\_ids = nullptr;
454
455     size\_t records\_bytes = num\_records * sizeof(double);
456
457     CUDA\_CHECK(cudaMalloc(&d\_v1, records\_bytes));
458     CUDA\_CHECK(cudaMalloc(&d\_v7, records\_bytes));
459     CUDA\_CHECK(cudaMalloc(&d\_v11, records\_bytes));
460     CUDA\_CHECK(cudaMalloc(&d\_amount, records\_bytes));
461     CUDA\_CHECK(cudaMalloc(&d\_period\_ids, num\_records * sizeof(int64\_t)
462 ));
463
464     CUDA\_CHECK(cudaMemcpy(d\_v1, h\_v1, records\_bytes,
465 cudaMemcpyHostToDevice));
466     CUDA\_CHECK(cudaMemcpy(d\_v7, h\_v7, records\_bytes,
467 cudaMemcpyHostToDevice));
468     CUDA\_CHECK(cudaMemcpy(d\_v11, h\_v11, records\_bytes,
469 cudaMemcpyHostToDevice));
470     CUDA\_CHECK(cudaMemcpy(d\_amount, h\_amount, records\_bytes,
471 cudaMemcpyHostToDevice));
472
473     double step1\_ms = get\_time\_ms() - step1\_start;

```

```

471
472
473 double step2\_start = get\_time\_ms();
474
475 const int BLOCK\_SIZE = 256;
476 int num\_blocks = (num\_records + BLOCK\_SIZE - 1) / BLOCK\_SIZE;
477
478 compute\_period\_ids\_kernel<<<num\_blocks, BLOCK\_SIZE>>>(
479     d\_v1, d\_period\_ids, num\_records, interval);
480 CUDA\_CHECK(cudaGetLastError());
481 CUDA\_CHECK(cudaDeviceSynchronize());
482
483 double step2\_ms = get\_time\_ms() - step2\_start;
484
485
486
487
488 double step3\_start = get\_time\_ms();
489
490
491 int num\_periods = 0;
492 int64\_t* d\_unique\_periods = nullptr;
493 int* d\_counts = nullptr;
494 int* d\_offsets = nullptr;
495
496
497 int* d\_indices = nullptr;
498 int64\_t* d\_sorted\_period\_ids = nullptr;
499 CUDA\_CHECK(cudaMalloc(&d\_indices, num\_records * sizeof(int)));
500 CUDA\_CHECK(cudaMalloc(&d\_sorted\_period\_ids, num\_records * sizeof(
int64\_t)));
501
502
503 int64\_t* h\_period\_ids = nullptr;
504 CUDA\_CHECK(cudaMallocHost(&h\_period\_ids, num\_records * sizeof(int64
\_t)));
505
506 CUDA\_CHECK(cudaMemcpy(h\_period\_ids, d\_period\_ids, num\_records *
sizeof(int64\_t),
507                      cudaMemcpyDeviceToHost));
508
509
510 int64\_t min\_period = h\_period\_ids[0];
511 int64\_t max\_period = h\_period\_ids[0];
512 #pragma omp parallel for reduction(min:min\_period) reduction(max:max\_
period)
513 for (int i = 1; i < num\_records; i++) {
514     if (h\_period\_ids[i] < min\_period) min\_period = h\_period\_ids[i
];
515     if (h\_period\_ids[i] > max\_period) max\_period = h\_period\_ids[i
];
516 }
517
518 int64\_t period\_range = max\_period - min\_period + 1;
519
520
521 if (period\_range > 0 && period\_range < 10000) {
522
523     int* count = new int[period\_range]();
524     int* indices\_sorted = new int[num\_records];
525

```

```

526
527 #pragma omp parallel for
528 for (int i = 0; i < num\_records; i++) {
529     int period\_idx = (int)(h\_period\_ids[i] - min\_period);
530     #pragma omp atomic
531     count[period\_idx]++;
532 }
533
534
535 int* offsets = new int[period\_range];
536 offsets[0] = 0;
537 for (int i = 1; i < period\_range; i++) {
538     offsets[i] = offsets[i-1] + count[i-1];
539 }
540
541
542 int* temp\_offsets = new int[period\_range];
543 memcpy(temp\_offsets, offsets, period\_range * sizeof(int));
544
545
546 for (int i = 0; i < num\_records; i++) {
547     int period\_idx = (int)(h\_period\_ids[i] - min\_period);
548     indices\_sorted[temp\_offsets[period\_idx]++] = i;
549 }
550
551
552 CUDA\_CHECK(cudaMemcpy(d\_indices, indices\_sorted, num\_records *
sizeof(int),
553                                     cudaMemcpyHostToDevice));
554
555
556 copy\_sorted\_period\_ids\_kernel<<<num\_blocks, BLOCK\_SIZE>>>(
557     d\_period\_ids, d\_indices, d\_sorted\_period\_ids, num\_records
);
558 CUDA\_CHECK(cudaGetLastError());
559 CUDA\_CHECK(cudaDeviceSynchronize());
560
561
562 int num\_periods = 0;
563 int64\_t* h\_unique\_periods = new int64\_t[period\_range];
564 int* h\_counts = new int[period\_range];
565 int* h\_offsets = new int[period\_range];
566
567 for (int i = 0; i < period\_range; i++) {
568     if (count[i] > 0) {
569         h\_unique\_periods[num\_periods] = min\_period + i;
570         h\_counts[num\_periods] = count[i];
571         h\_offsets[num\_periods] = offsets[i];
572         num\_periods++;
573     }
574 }
575
576 delete[] count;
577 delete[] offsets;
578 delete[] temp\_offsets;
579 delete[] indices\_sorted;
580
581
582 CUDA\_CHECK(cudaMalloc(&d\_unique\_periods, num\_periods * sizeof(
int64\_t)));
583 CUDA\_CHECK(cudaMalloc(&d\_counts, num\_periods * sizeof(int)));

```

```

584     CUDA\_CHECK(cudaMalloc(&d\_offsets, num\_periods * sizeof(int)));
585
586     CUDA\_CHECK(cudaMemcpy(d\_unique\_periods, h\_unique\_periods,
587         num\_periods * sizeof(int64\_t),
588         cudaMemcpyHostToDevice));
589     CUDA\_CHECK(cudaMemcpy(d\_counts, h\_counts,
590         num\_periods * sizeof(int),
591         cudaMemcpyHostToDevice));
592     CUDA\_CHECK(cudaMemcpy(d\_offsets, h\_offsets,
593         num\_periods * sizeof(int),
594         cudaMemcpyHostToDevice));
595
596     delete[] h\_unique\_periods;
597     delete[] h\_counts;
598     delete[] h\_offsets;
599     CUDA\_CHECK(cudaFreeHost(h\_period\_ids));
600
601 } else {
602
603     PeriodIndexPair* pairs = new PeriodIndexPair[num\_records];
604     for (int i = 0; i < num\_records; i++) {
605         pairs[i].period = h\_period\_ids[i];
606         pairs[i].index = i;
607     }
608
609     qsort(pairs, num\_records, sizeof(PeriodIndexPair), compare\_period\_
610 _pairs);
611
612     int64\_t* h\_unique\_periods = new int64\_t[num\_records];
613     int* h\_counts = new int[num\_records];
614     int* h\_offsets = new int[num\_records];
615
616     if (num\_records > 0) {
617         int64\_t current\_period = pairs[0].period;
618         h\_unique\_periods[0] = current\_period;
619         h\_offsets[0] = 0;
620         h\_counts[0] = 1;
621         num\_periods = 1;
622
623         for (int i = 1; i < num\_records; i++) {
624             if (pairs[i].period == current\_period) {
625                 h\_counts[num\_periods - 1]++;
626             } else {
627                 current\_period = pairs[i].period;
628                 h\_unique\_periods[num\_periods] = current\_period;
629                 h\_offsets[num\_periods] = i;
630                 h\_counts[num\_periods] = 1;
631                 num\_periods++;
632             }
633         }
634     }
635
636     int* h\_indices = new int[num\_records];
637     for (int i = 0; i < num\_records; i++) {
638         h\_indices[i] = pairs[i].index;
639     }
640

```

```

641     CUDA\_CHECK(cudaMemcpy(d\_indices, h\_indices, num\_records * sizeof
(int),
642                               cudaMemcpyHostToDevice));
643
644
645     copy\_sorted\_period\_ids\_kernel<<<num\_blocks, BLOCK\_SIZE>>>(
646         d\_period\_ids, d\_indices, d\_sorted\_period\_ids, num\_records
);
647     CUDA\_CHECK(cudaGetLastError());
648     CUDA\_CHECK(cudaDeviceSynchronize());
649
650
651     CUDA\_CHECK(cudaMalloc(&d\_unique\_periods, num\_periods * sizeof(
int64\_t)));
652     CUDA\_CHECK(cudaMalloc(&d\_counts, num\_periods * sizeof(int));
653     CUDA\_CHECK(cudaMalloc(&d\_offsets, num\_periods * sizeof(int));
654
655     CUDA\_CHECK(cudaMemcpy(d\_unique\_periods, h\_unique\_periods,
656                             num\_periods * sizeof(int64\_t),
cudaMemcpyHostToDevice));
657     CUDA\_CHECK(cudaMemcpy(d\_counts, h\_counts,
658                             num\_periods * sizeof(int),
cudaMemcpyHostToDevice));
659     CUDA\_CHECK(cudaMemcpy(d\_offsets, h\_offsets,
660                             num\_periods * sizeof(int),
cudaMemcpyHostToDevice));
661
662     delete[] pairs;
663     delete[] h\_indices;
664     delete[] h\_unique\_periods;
665     delete[] h\_counts;
666     delete[] h\_offsets;
667     delete[] h\_period\_ids;
668 }
669
670 double step3\_ms = get\_time\_ms() - step3\_start;
671
672
673
674
675 double step3\_5\_start = get\_time\_ms();
676
677
678 double* d\_v1\_sorted = nullptr;
679 double* d\_v7\_sorted = nullptr;
680 double* d\_v11\_sorted = nullptr;
681 double* d\_amount\_sorted = nullptr;
682 CUDA\_CHECK(cudaMalloc(&d\_v1\_sorted, records\_bytes));
683 CUDA\_CHECK(cudaMalloc(&d\_v7\_sorted, records\_bytes));
684 CUDA\_CHECK(cudaMalloc(&d\_v11\_sorted, records\_bytes));
685 CUDA\_CHECK(cudaMalloc(&d\_amount\_sorted, records\_bytes));
686
687 reorder\_data\_kernel<<<num\_blocks, BLOCK\_SIZE>>>(
688     d\_v1, d\_v7, d\_v11, d\_amount,
689     d\_indices,
690     d\_v1\_sorted, d\_v7\_sorted, d\_v11\_sorted, d\_amount\_sorted,
691     num\_records);
692     CUDA\_CHECK(cudaGetLastError());
693     CUDA\_CHECK(cudaDeviceSynchronize());
694
695     cudaFree(d\_sorted\_period\_ids);

```

```

696     cudaFree(d\_indices);
697
698     double step3\_5\_ms = get\_time\_ms() - step3\_5\_start;
699
700
701
702
703     double step4\_start = get\_time\_ms();
704
705     GpuPeriodStats* d\_out\_stats = nullptr;
706     CUDA\_CHECK(cudaMalloc(&d\_out\_stats, num\_periods * sizeof(
GpuPeriodStats)));
707
708
709     const char* env\_block\_kernel = getenv("USE\_BLOCK\_KERNEL");
710     if (env\_block\_kernel == nullptr) {
711         printf("Error: Environment variable USE\_BLOCK\_KERNEL is not set\n"
);
712         return -1;
713     }
714     bool use\_block\_kernel = (atoi(env\_block\_kernel) != 0);
715
716     if (use\_block\_kernel) {
717
718         aggregate\_periods\_kernel<<<num\_periods, BLOCK\_SIZE>>>(
719             d\_v1\_sorted, d\_v7\_sorted, d\_v11\_sorted, d\_amount\_sorted,
720             d\_unique\_periods, d\_offsets, d\_counts,
721             num\_periods, d\_out\_stats);
722     } else {
723
724         int agg\_blocks = (num\_periods + BLOCK\_SIZE - 1) / BLOCK\_SIZE;
725         aggregate\_periods\_simple\_kernel<<<agg\_blocks, BLOCK\_SIZE>>>(
726             d\_v1\_sorted, d\_v7\_sorted, d\_v11\_sorted, d\_amount\_sorted,
727             d\_unique\_periods, d\_offsets, d\_counts,
728             num\_periods, d\_out\_stats);
729     }
730
731     CUDA\_CHECK(cudaGetLastError());
732     CUDA\_CHECK(cudaDeviceSynchronize());
733
734     double step4\_ms = get\_time\_ms() - step4\_start;
735
736
737
738
739     double step5\_start = get\_time\_ms();
740
741     GpuPeriodStats* h\_stats = new GpuPeriodStats[num\_periods];
742     CUDA\_CHECK(cudaMemcpy(h\_stats, d\_out\_stats, num\_periods * sizeof(
GpuPeriodStats),
743                         cudaMemcpyDeviceToHost));
744
745     double step5\_ms = get\_time\_ms() - step5\_start;
746
747
748
749
750     double step6\_start = get\_time\_ms();
751
752     cudaFree(d\_v1);
753     cudaFree(d\_v7);

```



```

754     cudaFree(d\_v11);
755     cudaFree(d\_amount);
756     cudaFree(d\_period\_ids);
757     cudaFree(d\_unique\_periods);
758     cudaFree(d\_counts);
759     cudaFree(d\_offsets);
760     cudaFree(d\_v1\_sorted);
761     cudaFree(d\_v7\_sorted);
762     cudaFree(d\_v11\_sorted);
763     cudaFree(d\_amount\_sorted);
764     cudaFree(d\_out\_stats);
765
766     double step6\_ms = get\_time\_ms() - step6\_start;
767
768
769
770
771     double total\_ms = get\_time\_ms() - total\_start;
772
773
774     printf(" GPU aggregation (%d records, interval=%.1f, kernel=%s):\n",
775           num\_records, interval, use\_block\_kernel ? "block" : "simple");
776     printf(" 1. Malloc + H->D copy: %7.3f ms\n", step1\_ms);
777     printf(" 2. Compute period\_ids: %7.3f ms\n", step2\_ms);
778     printf(" 3. Sort + group : %7.3f ms (%d periods)\n", step3\_ms, num\_
779 _periods);
780     printf(" 3.5. Reorder data (GPU): %7.3f ms\n", step3\_5\_ms);
781     printf(" 4. Aggregation kernel: %7.3f ms (%s)\n", step4\_ms, use\_
782 _block\_kernel ? "block" : "simple");
783     printf(" 5. D->H copy: %7.3f ms\n", step5\_ms);
784     printf(" 6. Free GPU memory: %7.3f ms\n", step6\_ms);
785     printf(" GPU TOTAL: %7.3f ms\n", total\_ms);
786     fflush(stdout);
787
788     *h\_out\_stats = h\_stats;
789     *out\_num\_periods = num\_periods;
790
791     return 0;
792 }
793
794
795
796 extern "C" void gpu\_free\_results(GpuPeriodStats* stats) {
797     delete[] stats;
798 }

```

Приложение Н

Исходный код файла утилит для работы.

Листинг 9: utils.cpp

```
1      #include "utils.hpp"
2      #include <fstream>
3      #include <sstream>
4      #include <stdexcept>
5      #include <numeric>
6      #include <algorithm>
7      #include <cstring>
8
9      int get\_num\_cpu\_threads() {
10         const char* env\_threads = std::getenv("NUM\_CPU\_THREADS");
11         int num\_cpu\_threads = 1;
12         if (env\_threads) {
13             num\_cpu\_threads = std::atoi(env\_threads);
14             if (num\_cpu\_threads < 1) num\_cpu\_threads = 1;
15         }
16         return num\_cpu\_threads;
17     }
18
19     std::string get\_env(const char* name) {
20         const char* env = std::getenv(name);
21         if (!env) {
22             throw std::runtime\_error(std::string("Environment variable not set:
23 ") + name);
24         }
25         return std::string(env);
26     }
27
28     std::string get\_data\_path() {
29         return get\_env("DATA\_PATH");
30     }
31
32     bool is\_gpu\_node(const std::string& node\_name) {
33         std::string lower\_name = node\_name;
34         std::transform(lower\_name.begin(), lower\_name.end(), lower\_name.begin
35 (), ::tolower);
36         return lower\_name.find("gpu") != std::string::npos;
37     }
38
39     double get\_gpu\_process\_weight() {
40         const char* env = std::getenv("GPU\_PROCESS\_WEIGHT");
41         if (env) {
42             try {
43                 double weight = std::stod(env);
44                 if (weight > 0.0) {
45                     return weight;
46                 }
47             } catch (...) {
48             }
49         }
50
51         return 1.28;
52     }
53 }
```

```

54 double get\_cpu\_process\_weight() {
55     const char* env = std::getenv("CPU\_PROCESS\_WEIGHT");
56     if (env) {
57         try {
58             double weight = std::stod(env);
59             if (weight > 0.0) {
60                 return weight;
61             }
62         } catch (...) {
63
64         }
65     }
66
67     return 1.0;
68 }
69
70 std::vector<double> calculate\_process\_weights(const std::vector<bool>& process\_has\_gpu) {
71     double gpu\_weight = get\_gpu\_process\_weight();
72     double cpu\_weight = get\_cpu\_process\_weight();
73
74     std::vector<double> weights;
75     weights.reserve(process\_has\_gpu.size());
76
77     for (bool has\_gpu : process\_has\_gpu) {
78         weights.push\_back(has\_gpu ? gpu\_weight : cpu\_weight);
79     }
80
81     return weights;
82 }
83
84 std::vector<int> weights\_to\_shares(const std::vector<double>& weights,
85     int base\_share) {
86     if (weights.empty()) {
87         return std::vector<int>();
88     }
89
90     double min\_weight = *std::min\_element(weights.begin(), weights.end());
91     if (min\_weight <= 0.0) {
92         min\_weight = 1.0;
93     }
94
95
96     double total\_weight = 0.0;
97     for (double w : weights) {
98         total\_weight += w;
99     }
100
101     if (total\_weight <= 0.0) {
102
103         return std::vector<int>(weights.size(), base\_share);
104     }
105
106
107     std::vector<int> shares;
108     shares.reserve(weights.size());
109
110     for (double w : weights) {
111
112

```

```

113         double normalized = (w / min\_weight) * base\_share;
114         shares.push\_back(static\_cast<int>(normalized + 0.5));
115     }
116
117     return shares;
118 }
119
120 std::vector<int> calculate\_data\_shares\_auto(int num\_nodes, const std::
vector<std::string>& node\_names) {
121     std::vector<int> shares;
122
123
124     if (node\_names.empty() || node\_names.size() != static\_cast<size\_t>(
num\_nodes)) {
125         shares.assign(num\_nodes, 10);
126         return shares;
127     }
128
129
130     int num\_gpu = 0;
131     int num\_cpu = 0;
132     std::vector<bool> is\_gpu(num\_nodes, false);
133     std::vector<int> gpu\_indices;
134     std::vector<int> cpu\_indices;
135
136     for (size\_t i = 0; i < node\_names.size(); i++) {
137         if (is\_gpu\_node(node\_names[i])) {
138             is\_gpu[i] = true;
139             num\_gpu++;
140             gpu\_indices.push\_back(static\_cast<int>(i));
141         } else {
142             num\_cpu++;
143             cpu\_indices.push\_back(static\_cast<int>(i));
144         }
145     }
146
147
148
149
150
151
152     const int base\_cpu\_share = 10;
153     const double gpu\_performance\_ratio = 1.286;
154     const int base\_gpu\_share = static\_cast<int>(base\_cpu\_share * gpu\_
performance\_ratio + 0.5);
155
156
157     if (num\_gpu > 0 && num\_cpu > 0) {
158         shares.resize(num\_nodes);
159
160
161         if (num\_cpu == 1) {
162             shares[cpu\_indices[0]] = base\_cpu\_share;
163         } else if (num\_cpu == 2) {
164             shares[cpu\_indices[0]] = base\_cpu\_share;
165             shares[cpu\_indices[1]] = base\_cpu\_share + 1;
166         } else {
167
168             for (size\_t i = 0; i < cpu\_indices.size(); i++) {
169                 shares[cpu\_indices[i]] = base\_cpu\_share + (i % 2);
170             }
171

```

```

171     }
172
173
174     if (num\_gpu == 1) {
175         shares[gpu\_indices[0]] = base\_gpu\_share;
176     } else if (num\_gpu == 2) {
177         shares[gpu\_indices[0]] = base\_gpu\_share;
178         shares[gpu\_indices[1]] = base\_gpu\_share + 1;
179     } else {
180
181         for (size\_t i = 0; i < gpu\_indices.size(); i++) {
182             shares[gpu\_indices[i]] = base\_gpu\_share + (i % 2);
183         }
184     }
185 } else if (num\_gpu > 0) {
186
187     shares.assign(num\_nodes, base\_gpu\_share);
188 } else {
189
190     shares.assign(num\_nodes, base\_cpu\_share);
191 }
192
193 return shares;
194 }
195
196 std::vector<int> get\_data\_read\_shares() {
197
198     const char* env\_shares = std::getenv("DATA\_READ\_SHARES");
199     if (env\_shares && strlen(env\_shares) > 0) {
200         std::vector<int> shares;
201         std::stringstream ss(env\_shares);
202         std::string item;
203         while (std::getline(ss, item, ',')) {
204             shares.push\_back(std::stoi(item));
205         }
206         return shares;
207     }
208
209
210     const char* nodelist = std::getenv("SLURM\_JOB\_NODELIST");
211     const char* ntasks = std::getenv("SLURM\_NTASKS");
212     const char* nnodes = std::getenv("SLURM\_NNODES");
213
214
215     int num\_tasks = 0;
216     if (ntasks) {
217         num\_tasks = std::atoi(ntasks);
218     } else if (nnodes) {
219         num\_tasks = std::atoi(nnodes);
220     }
221
222     if (num\_tasks > 0) {
223         std::vector<std::string> node\_names;
224
225
226         if (nodelist) {
227             std::string nodelist\_str(nodelist);
228
229
230             std::stringstream ss(nodelist\_str);
231             std::string node;

```

```

232         while (std::getline(ss, node, ',')) {
233
234             node.erase(std::remove_if(node.begin(), node.end(), ::
isspace), node.end());
235             if (!node.empty()) {
236
237                 size_t bracket = node.find('[');
238                 if (bracket != std::string::npos) {
239                     node = node.substr(0, bracket);
240                 }
241                 node\_names.push\_back(node);
242             }
243         }
244     }
245
246
247     if (!node\_names.empty() && node\_names.size() == static\_cast<size
\_t>(num\_tasks)) {
248         return calculate\_data\_shares\_auto(num\_tasks, node\_names);
249     } else if (!node\_names.empty()) {
250
251
252         std::vector<std::string> expanded\_names;
253         for (int i = 0; i < num\_tasks; i++) {
254             expanded\_names.push\_back(node\_names[i % node\_names.size
255             ()]);
256         }
257         return calculate\_data\_shares\_auto(num\_tasks, expanded\_names
258         );
259     } else {
260
261         return std::vector<int>(num\_tasks, 10);
262     }
263
264     return std::vector<int>(4, 10);
265 }
266
267 int64\_t get\_read\_overlap\_bytes() {
268     return std::stoll(get\_env("READ\_OVERLAP\_BYTES"));
269 }
270
271 int64\_t get\_aggregation\_interval() {
272     return std::stoll(get\_env("AGGREGATION\_INTERVAL"));
273 }
274
275 bool get\_use\_cuda() {
276     const char* env = std::getenv("USE\_CUDA");
277     if (!env) {
278         return false;
279     }
280     try {
281         return std::stoi(env) != 0;
282     } catch (...) {
283         return false;
284     }
285 }
286
287 int64\_t get\_file\_size(const std::string& path) {
288     std::ifstream file(path, std::ios::binary | std::ios::ate);

```

```

289     if (!file.is\_open()) {
290         throw std::runtime\_error("Cannot open file: " + path);
291     }
292     return static\_cast<int64\_t>(file.tellg());
293 }
294
295 ByteRange calculate\_byte\_range(int rank, int size, int64\_t file\_size,
296                                const std::vector<int>& shares, int64\_t
297                                overlap\_bytes) {
298     std::vector<int> effective\_shares;
299     if (shares.size() == static\_cast<size\_t>(size)) {
300         effective\_shares = shares;
301     } else {
302         effective\_shares.assign(size, 1);
303     }
304
305     int total\_shares = std::accumulate(effective\_shares.begin(), effective\_shares.end(), 0);
306     int64\_t bytes\_per\_share = file\_size / total\_shares;
307
308     int64\_t base\_start = 0;
309     for (int i = 0; i < rank; i++) {
310         base\_start += bytes\_per\_share * effective\_shares[i];
311     }
312
313     int64\_t base\_end = base\_start + bytes\_per\_share * effective\_shares[rank];
314
315     ByteRange range;
316
317     if (rank == 0) {
318         range.start = 0;
319         range.end = std::min(base\_end + overlap\_bytes, file\_size);
320     } else if (rank == size - 1) {
321         range.start = std::max(base\_start - overlap\_bytes, static\_cast<int64\_t>(0));
322         range.end = file\_size;
323     } else {
324         range.start = std::max(base\_start - overlap\_bytes, static\_cast<int64\_t>(0));
325         range.end = std::min(base\_end + overlap\_bytes, file\_size);
326     }
327
328     return range;
329 }
330
331 void trim\_edge\_periods(std::vector<PeriodStats>& periods, int rank, int
332 size) {
333     if (periods.empty()) return;
334
335     if (rank == 0) {
336         periods.pop\_back();
337     } else if (rank == size - 1) {
338         periods.erase(periods.begin());
339     } else {
340         periods.pop\_back();
341         periods.erase(periods.begin());
342     }
343 }

```

Приложение I

Исходный код файла структуры записи.

Листинг 10: record.hpp

```
1      #pragma once
2  #include <cstdint>
3
4  struct Record {
5      double v1;
6      double v7;
7      double v11;
8      double amount;
9  };
```


Приложение J

Исходный код файла структуры агрегированных данных.

Листинг 11: period_stats.hpp

```
1      #pragma once
2  #include <cstdint>
3
4  using PeriodIndex = int64_t;
5
6
7  struct PeriodStats {
8      PeriodIndex period;
9      double v1_min;
10     double v1_max;
11     double v11_sum;
12     int64_t negative_v7_count;
13     double amount_sum;
14     int64_t count;
15 };
```